

KAPITEL

13

Grafische Benutzungsoberflächen



Lernziele

Zentrale Konzepte in diesem Kapitel: Grafische Oberflächen konstruieren, Oberflächen-Layout, Oberflächen-Komponenten, Ereignisverarbeitung

Java-Konstrukte in diesem Kapitel: `JFrame`, `JLabel`, `JButton`, `JMenuBar`, `JMenu`, `JMenuItem`, `ActionEvent`, `Color`, `FlowLayout`, `BorderLayout`, `GridLayout`, `BoxLayout`, `Box`, `JOptionPane`, `EtchedBorder`, `EmptyBorder`, anonyme innere Klassen

13.1 Einführung

Bisher haben wir uns in diesem Buch darauf konzentriert, Anwendungen mit textbasierten Schnittstellen zu entwerfen. Der Grund dafür war nicht etwa, dass textbasierte Schnittstellen prinzipiell besser sind; sie sind lediglich einfacher zu konstruieren.

Wir wollten zu Beginn der Beschäftigung mit objektorientierter Programmierung nicht zu sehr die Aufmerksamkeit von den wichtigen Softwareentwicklungsprinzipien ablenken, zu denen wir den Aufbau von Objekten, ihre Interaktion, den Klassenentwurf und die Qualität von Quelltexten zählen.

Grafische Benutzungsoberflächen (im Folgenden meist kurz GUIs genannt, vom Englischen *Graphical User Interface*) werden auch mit interagierenden Objekten realisiert, haben aber eine sehr spezielle Struktur; wir haben ihre Einführung bisher vermieden, um zuerst die allgemeingültigen Prinzipien von Objektstrukturen zu besprechen. Nun sind wir jedoch ausreichend vorbereitet, um auch einen Blick auf die Konstruktion von grafischen Oberflächen zu werfen.

Wir werden in diesem Kapitel an einigen Stellen das Konzept der Lambda-Ausdrücke ausgiebig einsetzen, die mit Java 8 eingeführt wurden. Dieses Konzept haben wir ausführlich in Kapitel 5 besprochen – ein Kapitel, das wir an dieser Stelle im Buch als „fortgeschritten“ ausgewiesen hatten. Wenn Sie dieses Kapitel noch nicht gelesen haben, empfehlen wir Ihnen, dies jetzt nachzuholen, um sich mit der Syntax und dem Gebrauch von Lambda-Ausdrücken vertraut zu machen. Außerdem lohnt es sich, die als „fortgeschritten“ bezeichneten Themen in Kapitel 12 zu wiederholen, da in den GUI-Bibliotheken von Java in wesentlichem Umfang auf abstrakte Klassen und Interface-Typen zurückgegriffen wird.

Grafische Oberflächen geben unseren Anwendungen eine Schnittstelle, die aus Fenstern, Menüs, Knöpfen und anderen Komponenten besteht – und geben ihnen damit viel stärker das Aussehen von typischen Anwendungen, mit denen die meisten Computeranwender umgehen.

Beachten Sie, dass wir hier wieder über die Doppelbedeutung des Begriffs „Schnittstelle“ stolpern. Die Schnittstellen, über die wir hier reden, sind nicht die Schnittstellen von Klassen. Wir reden hier über *Benutzungsschnittstellen* – den Teil einer Anwendung, der auf dem Bildschirm sichtbar zur Interaktion mit dem Benutzer bereitsteht.

Sobald wir wissen, wie man GUIs mit Java erstellt, können wir sehr viel ansehnlichere Programme schreiben.

13.2 Komponenten, Layout und Ereignisbehandlung

Die Zahl der Detailmöglichkeiten bei der Konstruktion von GUIs ist riesig. Wir werden in diesem Buch nicht in der Lage sein, alle diese Details darzustellen, aber wir werden die grundlegenden Prinzipien und reichlich Beispiele diskutieren.

Sämtliche GUI-Programmierung in Java erfolgt durch die Benutzung von speziell dafür ausgewiesenen Standardbibliotheken. Sobald wir die Prinzipien verstanden haben, können wir alle notwendigen Details in der Dokumentation dieser Bibliotheken nachlesen.

Die Kernfragen, die wir klären müssen, ordnen sich thematisch in drei Bereiche:

- Welche Elemente können wir auf dem Bildschirm darstellen?
- Wie können wir diese Elemente auf dem Bildschirm arrangieren?
- Wie können wir auf Benutzereingaben reagieren?

Wir widmen uns diesen Fragen unter den Schlagworten *Komponenten*, *Layout* und *Ereignisverarbeitung*.

Komponenten sind die individuellen Bestandteile, aus denen sich eine grafische Oberfläche zusammensetzt. Dazu gehören Dinge wie Knöpfe, Menüs, Menüeinträge, Combo-Boxen, Schieberegler, Textfelder und so weiter. Die Java-Bibliothek enthält eine große Anzahl vorgefertigter Komponenten, und wir können auch eigene erstellen. Wir werden die wichtigsten Komponenten kennenlernen und erfahren, wie man sie erzeugt und so gestaltet, wie man sie gern hätte.

Das *Layout* regelt, wie die Komponenten auf dem Bildschirm arrangiert werden. Ältere, primitivere GUI-Systeme haben dies mit zweidimensionalen Koordinaten definiert: Ein Programmierer musste die x- und y-Koordinaten (in Bildschirmpunkten) für die Position und Größe jeder einzelnen Komponente festlegen. In moderneren GUI-Systemen wäre das zu kurz gedacht. Heutzutage müssen verschiedene Bildschirmauflösungen und Zeichensätze berücksichtigt werden, außerdem Benutzer, die die Fenster in ihrer Größe verändern, sowie weitere Aspekte, die dazu führen, dass Layout-Fragen sehr viel komplizierter werden. Die Lösung ist deshalb ein Schema, in dem das Layout von Komponenten auf einer abstrakteren Ebene beschrieben wird.

Konzept

Eine GUI wird erzeugt, indem **Komponenten** auf dem Bildschirm arrangiert werden. Komponenten werden durch Objekte repräsentiert.

Konzept

Das **Layout** der Komponenten, also ihre Anordnung auf dem Bildschirm, wird durch Layout-Manager kontrolliert.

Wir können darin beispielsweise festlegen, dass „diese Komponente unter jener erscheinen sollte“ oder dass „diese Komponente sich ausdehnen soll, wenn das Fenster vergrößert wird, aber jene immer ihre konstante Größe behalten sollte“. Wir werden sehen, dass dies mithilfe von sogenannten *Layout-Managern* möglich ist.

Ereignisverarbeitung wird die Technik genannt, mit der Benutzereingaben behandelt werden. Sobald wir die Komponenten unserer Oberfläche erzeugt und auf dem Bildschirm positioniert haben, sollten wir auch sicherstellen, dass etwas passiert, wenn ein Benutzer auf einen Knopf klickt. Das Modell, das in der Java-Bibliothek verwendet wird, basiert auf Ereignissen: Wenn ein Benutzer eine Komponente aktiviert (beispielsweise einen Knopf anklickt oder einen Menüeintrag selektiert), löst das System ein Ereignis aus. Unsere Anwendung kann dann von diesem Ereignis benachrichtigt werden (indem eine ihrer Methoden aufgerufen wird), und wir können die angemessene Reaktion generieren.

Wir werden diese Themenbereiche sehr viel detaillierter im Laufe dieses Kapitels untersuchen. Vorher brauchen wir jedoch noch ein wenig mehr Hintergrundinformation und etwas Terminologie.

Konzept

Der Begriff **Ereignisverarbeitung** bezeichnet die Aufgabe, auf Benutzereingaben wie Mausklicks oder Tastendrucke zu reagieren.

13.3 AWT und Swing

Java verfügt über drei GUI-Bibliotheken. Die älteste heißt *AWT* (Abstract Window Toolkit) und wurde mit dem allerersten Java-System ausgeliefert. Später wurde eine stark verbesserte GUI-Bibliothek namens *Swing* hinzugefügt. Zuletzt kam die Bibliothek *JavaFX* hinzu. In diesem Kapitel werden wir uns auf den Einsatz von Swing konzentrieren, doch viele der hier behandelten Prinzipien können allgemein auf GUI-Bibliotheken angewandt werden.

Swing benutzt einige der Klassen aus dem AWT, ersetzt einige AWT-Klassen mit einer eigenen Version und fügt sehr viele neue Klassen hinzu (Abbildung 13.1). Das bedeutet, dass wir einige AWT-Klassen verwenden, die noch immer in Swing-Programmen benutzt werden, aber dass wir die Swing-Versionen aller Klassen verwenden, die in beiden Bibliotheken angeboten werden.

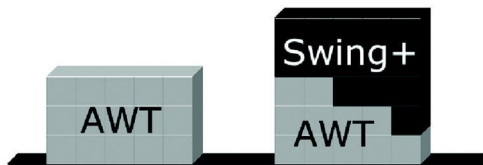


Abbildung 13.1

AWT und Swing.

Immer dann, wenn äquivalente Klassen in AWT und Swing existieren, sind die Swing-Versionen durch ein „J“ am Anfang des Klassennamens gekennzeichnet. Sie werden beispielsweise die Klassen `Button` und `JButton` finden, `Frame` und `JFrame`, `Menu` und `JMenu` und so weiter. Die Klassen mit dem „J“ am Anfang sind die Swing-Versionen – und diese werden wir benutzen. In einer Anwendung sollten die beiden Versionen nicht gemeinsam benutzt werden.

Das ist vorläufig genug Hintergrundinformation. Sehen wir uns einen Programmtext genauer an.

13.4 Das Beispiel: ein Bildbetrachter

Konzept

Bildformat: Digitale Bilder können in unterschiedlichen Formaten gespeichert werden. Die Unterschiede wirken sich primär auf die Größe und den Informationsgehalt aus.

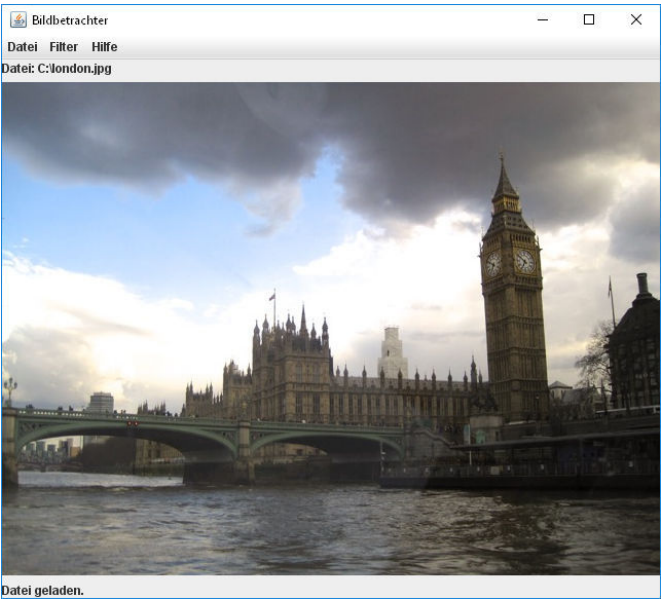
Wie immer werden wir uns die neuen Konzepte anhand eines Beispiels verdeutlichen. Die Anwendung, die wir in diesem Kapitel erstellen werden, ist ein Bildbetrachter (Abbildung 13.2). Mit diesem Programm können Bilddateien im JPEG- und im PNG-Format betrachtet werden, die Bilder können aber auch verändert und wieder auf der Platte gespeichert werden.

Insgesamt werden wir eine eigene Bildklasse zur Repräsentation von Bilddaten im Speicher benutzen, wir werden verschiedene Filter implementieren, mit denen wir ein Bild verändern können, und wir werden Swing-Komponenten benutzen, um eine grafische Oberfläche zu erstellen. Während wir all das tun, wollen wir uns primär mit der Diskussion der GUI-Aspekte beschäftigen.

Wenn Sie neugierig auf das sind, was wir bauen werden, dann können Sie das Projekt *Bildbetrachter1-0* öffnen und ausprobieren; erzeugen Sie einfach ein Objekt der Klasse **Bildbetrachter** – es ist die Version, die in Abbildung 13.2 gezeigt wird. Es gibt einige Beispielbilder im Ordner *bilder*, der sich im Ordner *Kapitel13* befindet (eine Ebene über dem Projektordner). Sie können natürlich auch Ihre eigenen Bilder öffnen. Wir werden hier jetzt etwas langsamer starten, mit einer viel simpleren Version, und uns den Weg zur vollständigen Anwendung Schritt für Schritt erarbeiten.

Abbildung 13.2

Eine einfache Anwendung zum Bildbetrachten.



13.4.1 Erste Schritte: ein Fenster erzeugen

Fast alles, was Sie in einer grafischen Oberfläche zu sehen bekommen, befindet sich in einem speziellen Fenster, einem *Top-Level-Fenster*. Ein solches Fenster heißt *Top-Level*, weil es sich unmittelbar unter der Kontrolle der Fensterverwaltung des jeweiligen Betriebssystems befindet und typischerweise unabhängig von anderen Fenstern bewegt, in der Größe verändert, minimiert und maximiert werden kann.

In Java heißen diese Top-Level-Fenster *Frames*. In Swing werden sie durch die Klasse `JFrame` repräsentiert.

```
import java.awt.*;
import java.awt.event.*;
import java.awt.image.*;
import javax.swing.*;

/**
 * Bildbetrachter ist die Hauptklasse der Bildbetrachter-Anwendung. Sie
 * erstellt die GUI der Anwendung, zeigt sie an und initialisiert alle
 * anderen Komponenten.
 *
 * Erzeugen Sie ein Exemplar dieser Klasse, um die Anwendung zu starten.
 *
 * @author Michael Kölling und David J. Barnes
 * @version 0.1
 */
public class Bildbetrachter
{
    private JFrame fenster;

    /**
     * Erzeuge einen Bildbetrachter und zeige seine GUI auf
     * dem Bildschirm an.
     */
    public Bildbetrachter()
    {
        fensterErzeugen();
    }

    // ---- Swing-Anteil zum Erzeugen des Fensters mit allen Komponenten ----

    /**
     * Erzeuge das Swing-Fenster samt Inhalt.
     */
    private void fensterErzeugen()
    {
        fenster = new JFrame("Bildbetrachter");
        Container contentPane = fenster.getContentPane();

        JLabel label = new JLabel("Ich bin ein Label. Ich kann Text anzeigen.");
        contentPane.add(label);

        fenster.pack();
        fenster.setVisible(true);
    }
}
```

Listing 13.1

Eine erste
Version der Klasse
Bildbetrachter.

Um eine grafische Oberfläche auf den Bildschirm zu bekommen, müssen wir als Erstes einen solchen Frame erzeugen und anzeigen. In Listing 13.1 wird die komplette Klassendefinition einer Klasse gezeigt (in weiser Vorausschau **Bildbetrachter** genannt), die einen Frame auf dem Bildschirm anzeigt. Diese Klasse ist im Projekt *Bildbetrachter0-1* verfügbar (die Nummer steht für Version 0.1).

Übung 13.1 Öffnen Sie das Projekt *Bildbetrachter0-1* (dies wird die Basis für Ihren eigenen Bildbetrachter). Erzeugen Sie eine Instanz der Klasse **Bildbetrachter**. Vergrößern Sie das angezeigte Fenster. Was beobachten Sie bei der Platzierung des Textes innerhalb des Fensters?

Wir werden uns nun die Klasse **Bildbetrachter** aus Listing 13.1 im Detail ansehen.

Die ersten drei Zeilen der Klassendefinition sind **import**-Anweisungen für alle Klassen aus den Paketen **java.awt**, **java.awt.event** und **javax.swing**¹. Wir brauchen etliche der Klassen in diesen Paketen für alle Swing-Anwendungen, deswegen sollten wir diese drei Pakete immer komplett in all unsere GUI-Anwendungen importieren.

Ein Blick auf den Rest der Klassendefinition zeigt schnell, dass die interessanten Dinge in der Methode **fensterErzeugen** passieren. Diese Methode kümmert sich um den Aufbau der grafischen Oberfläche. Im Konstruktor der Klasse steht lediglich ein Aufruf dieser Methode. Wir haben dies getan, weil auf diese Weise sämtlicher Quelltext zur Erzeugung der Oberfläche an einer wohldefinierten Stelle steht und später leichter wiederzufinden ist (Kohäsion!). Wir werden dies in all unseren GUI-Beispielen ebenfalls tun.

Die Klasse hat eine Instanzvariable vom Typ **JFrame**. Diese Variable hält eine Referenz auf das Fenster, das der Bildbetrachter auf dem Bildschirm anzeigen möchte. Sehen wir uns die Methode **fensterErzeugen** genauer an.

Die erste Zeile lautet:

```
fenster = new JFrame("Bildbetrachter");
```

Diese Anweisung erzeugt einen neuen Frame und speichert ihn in einer Instanzvariablen für die weitere Verwendung.

Als prinzipielle Richtlinie sollten Sie sich, parallel zum Studieren der Beispiele im Buch, auch die Klassendokumentationen der Klassen ansehen, die wir neu kennenlernen. Dies gilt für alle Klassen, die wir benutzen – wir werden von jetzt an nicht mehr darauf hinweisen, sondern davon ausgehen, dass Sie es selbstständig tun.

Übung 13.2 Finden Sie die Dokumentation der Klasse **JFrame**. Welchem Zweck dient der Parameter **Bildbetrachter**, den wir im obigen Konstruktoraufruf verwenden?

Konzept

Komponenten werden in einem Fenster platziert, indem sie entweder in die **Menüzeile** (*menu bar*) oder die **Inhaltsfläche** (*content pane*) eingefügt werden.

Ein Frame besteht aus drei Teilen: der *Titelzeile*, einer optionalen *Menüzeile* (*menu bar*) und der *Inhaltsfläche* (*content pane*), siehe Listing 13.3. Das exakte Erscheinungsbild der Titelzeile hängt vom jeweiligen Betriebssystem ab. Sie enthält normalerweise den Fenstertitel und einige Knöpfe zur Fensterkontrolle.

Die Menüzeile und die Inhaltsfläche befinden sich unter der Kontrolle der Anwendung. Beiden können GUI-Komponenten hinzugefügt werden, um eine Oberfläche zu konstruieren. Wir sehen uns zuerst die Inhaltsfläche an.

¹ Das **swing**-Paket befindet sich tatsächlich in einem Paket, das **javax** (mit einem „x“ am Ende) und nicht **java** heißt. Dies hat überwiegend historische Gründe – es scheint keine logische Erklärung dafür zu geben.

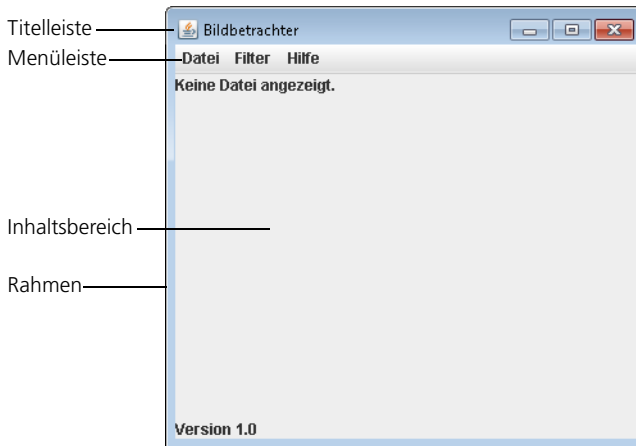


Abbildung 13.3
Die verschiedenen
Teile eines Frames.

13.4.2 Einfache Komponenten einfügen

Unmittelbar nach Erzeugung des **JFrame** ist das Fenster unsichtbar und die Inhaltsfläche ist leer. Wir fahren fort, indem wir einen Text in die Inhaltsfläche einfügen.

```
Container contentPane = frame.getContentPane();
```

```
JLabel label = new JLabel("Ich bin ein Label");
contentPane.add(label);
```

In der ersten Zeile lassen wir uns die Inhaltsfläche des Fensters geben. Dies müssen wir immer tun – GUI-Komponenten werden in einen Frame eingefügt, indem sie in die Inhaltsfläche eingefügt werden.²

Die Inhaltsfläche selbst ist vom Typ **Container**. Ein solcher Container ist eine Swing-Komponente, die eine beliebige Gruppe weiterer GUI-Komponenten enthalten kann – so wie eine **ArrayList** eine beliebige Sammlung von Objekten enthalten kann. Wir werden uns Container später noch genauer ansehen.

Wir erzeugen dann eine Komponente, die mit einem Text versehen werden kann (quasi ein „Etikett“, deshalb vom Typ **JLabel**) und fügen sie in die Inhaltsfläche ein. Ein solches Label kann Text und/oder ein Bild anzeigen.

Schließlich haben wir die beiden Zeilen

```
fenster.pack();
fenster.setVisible();
```

Die erste Zeile sorgt dafür, dass das Fenster die eingefügten Komponenten ordentlich arrangiert und sich selbst entsprechend in der Größe anpasst. Wir müssen die Methode **pack** jedes Mal an einem **JFrame** aufrufen, wenn wir Komponenten hinzufügen oder in der Größe verändern.

² Die Java-Klasse **JFrame** definiert ebenfalls eine **add**-Methode, mit der eine Komponente direkt in die Inhaltsfläche eingefügt werden kann (ohne den Umweg über **getContentPane()**). Da wir allerdings den expliziten Zugriff auf die Inhaltsfläche aus bestimmten Gründen später sowieso noch benötigen, können wir besagte Technik ebenso gut auch hier schon zum Einfügen der Komponenten in die Inhaltsfläche verwenden.

Die letzte Zeile schließlich macht das Fenster auf dem Bildschirm sichtbar. Wir beginnen immer mit einem unsichtbaren Fenster, damit das Arrangieren der Komponenten innerhalb des Fensters während des Konstruktionsprozesses nicht sichtbar wird. Wenn das Fenster vollständig konstruiert ist, können wir es im kompletten Zustand anzeigen.

Übung 13.3 Eine weitere häufig verwendete Swing-Komponente ist ein Knopf (*button*, Typ `JButton`), der gedrückt werden kann. Ersetzen Sie im obigen Beispiel das Label mit einem Knopf.

Übung 13.4 Was passiert, wenn Sie zwei Label (oder zwei Knöpfe) in die Inhaltsfläche einfügen? Können Sie erklären, was Sie beobachten? Experimentieren Sie mit Größenänderungen am Fenster.

13.4.3 Eine alternative Struktur

Wir haben uns dafür entschieden, unsere Anwendung zu entwickeln, indem wir ein `JFrame`-Objekt als Attribut der Klasse `Bildbetrachter` erzeugen und es mit weiteren GUI-Komponenten bestücken, die außerhalb des Frame-Objekts erzeugt werden. Eine alternative Struktur für diese Top-Level-Lösung wäre es, `Bildbetrachter` als eine Subklasse von `JFrame` zu definieren und intern zu bestücken. Dieser Stil ist ebenfalls häufig anzutreffen. Listing 13.2 ist die alternative Entsprechung zu Listing 13.1. Diese Version finden Sie als Projekt unter der Bezeichnung *Bildbetrachter0-1a*. Sie sollten sich mit beiden Stilen vertraut machen, denn keine ist der anderen vorzuziehen. Wir werden im Rest des Kapitels mit unserer ursprünglichen Version fortfahren.

Listing 13.2

Eine alternative Struktur für die Klasse `Bildbetrachter`.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

/**
 * Bildbetrachter ist die Hauptklasse der Bildbetrachter-Anwendung. Sie
 * erstellt die GUI der Anwendung, zeigt sie an und initialisiert alle
 * anderen Komponenten.
 *
 * Erzeugen Sie ein Exemplar dieser Klasse, um die Anwendung zu starten.
 *
 * @author Michael Kölling und David J. Barnes
 * @version 0.1a
 */
public class Bildbetrachter extends JFrame
{
    /**
     * Erzeuge einen Bildbetrachter und zeige seine GUI auf
     * dem Bildschirm an.
     */
    public Bildbetrachter()
    {
        super("Bildbetrachter");
        fensterErzeugen();
    }

    // ---- Swing-Anteil zum Erzeugen des Fensters mit allen Komponenten ----
}
```

```

/**
 * Erzeuge das Swing-Fenster samt Inhalt.
 */
private void fensterErzeugen()
{
    Container contentPane = getContentPane();

    JLabel label = new JLabel("Ich bin ein Label. Ich kann Text anzeigen.");
    contentPane.add(label);

    pack();
    setVisible(true);
}
}

```

13.4.4 Menüs hinzufügen

Unser nächster Schritt bei der Konstruktion einer GUI ist das Definieren von Menüs und Menüeinträgen. Konzeptuell ist das einfach, mit einem schwierigen Detail: Wie definieren wir die Reaktion auf Benutzereingaben, beispielsweise die Auswahl eines Menüeintrags? Dazu kommen wir gleich.

Zuerst erstellen wir die Menüs. Drei Klassen sind daran beteiligt:

- **JMenuBar** – Ein Objekt dieser Klasse repräsentiert eine Menüleiste, die unterhalb der Titelleiste am oberen Rand eines Fensters angezeigt werden kann (siehe Abbildung 13.3). Jedes Fenster hat höchstens einen **JMenuBar**.³
- **JMenu** – Objekte dieser Klasse repräsentieren ein einzelnes Menü (wie etwa die bekannten Menüs „Datei“, „Bearbeiten“ oder „Hilfe“). Menüs werden oft in einer Menüleiste platziert. Sie können auch als Kontextmenüs auftreten, aber diese werden wir vorläufig nicht behandeln.
- **JMenuItem** – Objekte dieser Klasse repräsentieren einen einzelnen Menüeintrag innerhalb eines Menüs (wie etwa „Öffnen“ oder „Speichern“).

Für unseren Bildbetrachter werden wir eine Menüleiste mit mehreren Menüs und Menüeinträgen erzeugen.

Die Klasse **JFrame** bietet eine Methode **setJMenuBar**. Wir können eine Menüleiste erzeugen und sie mit dieser Methode in das Hauptfenster eintragen:

```

JMenuBar menuezeile = new JMenuBar();
fenster.setJMenuBar(menuezeile);

```

Nun können wir ein Menü erzeugen und es in die Menüleiste einfügen:

```

JMenu dateimenue = new JMenu("Datei");
menuezeile.add(dateimenue);

```

Diese beiden Zeilen erzeugen ein Menü mit der Bezeichnung DATEI und fügen es in die Menüleiste ein. Schließlich können wir Menüeinträge in das Menü einfügen. Die folgenden Zeilen fügen zwei Einträge, bezeichnet mit **ÖFFNEN** und **BEENDEN**, in das Dateimenü ein:

³ Im MacOS ist die Standarddarstellung anders: Die Menüleiste befindet sich am oberen Rand des Bildschirms, nicht am oberen Rand des jeweiligen Fensters. In Java-Anwendungen sorgt die Voreinstellung dafür, dass die Menüleiste Teil des Fensters ist. Sie kann über eine für das MacOS spezifische Einstellungsmöglichkeit auch bei Java-Anwendungen am oberen Bildschirmrand platziert werden.

```
JMenuItem oeffnenEintrag = new JMenuItem("Öffnen");
dateiMenue.add(oeffnenEintrag);
JMenuItem beendenEintrag = new JMenuItem("Beenden");
dateiMenue.add(beendenEintrag);
```

Übung 13.5 Fügen Sie das hier diskutierte Menü und die Menüeinträge in Ihr *Bildbetrachter*-Projekt ein. Was passiert, wenn Sie einen Menüeintrag selektieren?

Übung 13.6 Fügen Sie ein weiteres Menü *HILFE* hinzu, das einen Eintrag *INFO* enthält. (*Hinweis:* Um Lesbarkeit und Kohäsion zu erhöhen, könnte es sinnvoll sein, die Erzeugung der Menüs in eine eigene Methode mit einem Namen wie `menuezeileErzeugen` zu verschieben, die aus der Methode `fensterErzeugen` aufgerufen wird.)

Bis zu diesem Punkt haben wir bereits die halbe Miete: Wir können Menüs erzeugen und anzeigen. Aber die zweite Hälfte fehlt noch: Bisher passiert nichts, wenn ein Benutzer einen Menüeintrag auswählt. Wir müssen nun Code hinzufügen, mit dem auf die Selektion von Menüeinträgen reagiert werden kann. Dies ist Thema des nächsten Abschnitts.

13.4.5 Ereignisverarbeitung

Swing benutzt ein sehr flexibles Modell für den Umgang mit Oberflächeneingaben: Die *Ereignisverarbeitung* mit sogenannten *Event-Listern*, die quasi auf Ereignisse „lauschen“ (vom Englischen *to listen*, zuhören).

Konzept

Ein Objekt kann auf Ereignisse „lauschen“, indem es ein **Event-Listener**-Interface implementiert.

Das Swing-Rahmenwerk selbst und einige seiner Komponenten lösen Ereignisse (*events*) aus, wenn etwas passiert, an dem andere Objekte interessiert sein könnten. Es gibt verschiedene Arten von Ereignissen, die durch unterschiedliche Arten von Aktionen ausgelöst werden. Wenn ein Knopf geklickt oder ein Menüeintrag selektiert wird, löst eine Komponente ein **ActionEvent** aus. Wenn eine Maus geklickt oder bewegt wird, wird ein **MouseEvent** ausgelöst. Wenn ein Fenster geschlossen oder minimiert wird, wird ein **WindowEvent** generiert. Darüber hinaus gibt es viele weitere Ereignisarten.

Jedes unserer Objekte kann zu einem Event-Listener für jedes dieser Ereignisse gemacht werden. Wenn es ein solcher Listener ist, wird es über alle Ereignisse informiert, auf die es „lauscht“. Ein Objekt wird zu einem Event-Listener, indem es eines der zahlreich existierenden Listener-Interfaces implementiert. Wenn es das richtige Interface implementiert, kann es sich bei der Komponente registrieren, die es „belauschen“ will.

Sehen wir uns ein Beispiel an. Menüeinträge (Klasse **JMenuItem**) lösen **ActionEvents** aus, wenn sie vom Benutzer selektiert werden. Objekte, die auf diese Ereignisse

reagieren wollen, müssen das Interface **ActionListener** aus dem Paket **java.awt.event** implementieren.

Es gibt zwei Möglichkeiten für die Implementierung von Event-Listnern: Entweder lauscht ein einzelnes Objekt auf Ereignisse von vielen verschiedenen Ereignisquellen oder jede Ereignisquelle bekommt einen eigenen Listener. Wir werden beide Möglichkeiten in den nächsten beiden Abschnitten diskutieren.

13.4.6 Die zentralisierte Variante für Ereignisverarbeitung

Wir müssen drei Dinge tun, um unseren Bildbetrachter zum einzigen Listener für alle Ereignisse aus dem Menü zu machen:

- 1** Wir müssen im Kopf der Klasse deklarieren, dass sie das Interface **ActionListener** implementiert.
- 2** Wir müssen eine Methode mit der Signatur

```
public void actionPerformed(ActionEvent e)
```

implementieren. Dies ist die einzige Methode, die im Interface **ActionListener** definiert ist.
- 3** Wir müssen die Methode **addActionListener** des jeweiligen Menüeintrags aufrufen, um das **Bildbetrachter**-Objekt als einen Listener zu registrieren.

Die Punkte 1 und 2 – Implementieren des Interface und Definieren der Methode – stellen sicher, dass unser Objekt ein Subtyp von **ActionListener** ist. Punkt 3 registriert unser eigenes Objekt dann als Listener bei den Menüeinträgen. Listing 13.3 zeigt den Quelltext für dieses Vorgehen im Zusammenhang.

```
public class Bildbetrachter
    implements ActionListener
{
    // Datenfelder und Konstruktor hier ausgelassen

    /**
     * Empfange die Benachrichtigung über eine Aktion.
     * @param event die Details der Aktion
     */
    public void actionPerformed(ActionEvent event)
    {
        System.out.println("Menüeintrag: " + event.getActionCommand());
    }

    /**
     * Erzeuge das Swing-Fenster samt Inhalt.
     */
    private void fensterErzeugen()
    {
        fenster = new JFrame("Bildbetrachter");
        menuzeileErzeugen(fenster);

        // Rest der GUI-Konstruktion hier ausgelassen
    }
}
```

Listing 13.3

Einfügen eines ActionListener bei einem Menüeintrag.

```

/**
 * Die Menüzeile des Hauptfensters erzeugen.
 * @param fenster das Fenster, in das die Menüzeile eingefügt werden soll
 */
private void menuezeileErzeugen(JFrame fenster)
{
    JMenuBar menuezeile = new JMenuBar();
    fenster.setJMenuBar(menuezeile);

    // Das Datei-Menü erzeugen
    JMenu dateiMenue = new JMenu("Datei");
    menuezeile.add(dateiMenue);

    JMenuItem oeffnenEintrag = new JMenuItem("Öffnen");
    oeffnenEintrag.addActionListener(this);
    dateiMenue.add(oeffnenEintrag);

    JMenuItem beendenEintrag = new JMenuItem("Beenden");
    beendenEintrag.addActionListener(this);
    dateiMenue.add(beendenEintrag);
}
}

```

Beachten Sie insbesondere die Zeilen

```

JMenuItem oeffnenEintrag = new JMenuItem("Öffnen");
oeffnenEintrag.addActionListener(this);

```

in diesem Programmbeispiel. Hier wird ein Menüeintrag erzeugt und anschließend das aktuelle Objekt (das **Bildbetrachter**-Objekt selbst) als Action-Listener registriert, indem **this** als Parameter an die Methode **addActionListener** übergeben wird.

Das Registrieren unseres Objekts als Listener des Menüeintrags bewirkt, dass unsere eigene Methode **actionPerformed** durch den Menüeintrag jedes Mal dann aufgerufen wird, wenn der Eintrag selektiert wird. Wenn unsere Methode aufgerufen wird, übergibt der Menüeintrag einen Parameter vom Typ **ActionEvent**, der zusätzliche Informationen über das eingetretene Ereignis liefert. Zu diesen Informationen gehören der exakte Zeitpunkt des Ereignisses, der Zustand der Zusatztasten (Umschalt-, Steuerung- und Alt/Meta-Tasten), ein „Kommando-String“ und einiges mehr.

Der Kommando-String ist eine Zeichenkette, die auf geeignete Weise die Komponente identifiziert, die das Ereignis ausgelöst hat. Bei Menüeinträgen ist dies per Voreinstellung der Text des Eintrags.

In unserem Beispiel in Listing 13.3 registrieren wir bei beiden Menüeinträgen dasselbe Listener-Objekt. Dies bedeutet, dass beide Menüeinträge dieselbe **actionPerformed**-Methode aufrufen, wenn sie aktiviert werden.

In der **actionPerformed**-Methode geben wir einfach den Kommando-String des Eintrags aus, um zu zeigen, dass dieses Schema funktioniert. An dieser Stelle könnten wir nun den Code einfügen, der die Selektion eines Menüeintrags korrekt verarbeitet.

Der bisher besprochene Quelltext steht als Projekt *Bildbetrachter0-2* im Zusatzmaterial zur Verfügung.

Übung 13.7 Implementieren Sie den oben diskutierten Code zur Verarbeitung von Menü-Ereignissen in Ihrem eigenen *Bildbetrachter*-Projekt. Alternativ können Sie auch das Projekt *Bildbetrachter0-2* öffnen und sich sorgfältig den Quelltext ansehen. Beschreiben Sie schriftlich und ausführlich die Sequenz der Ereignisse, die sich aus der Aktivierung des Menüeintrags **BEENDEN** ergibt.

Übung 13.8 Fügen Sie einen weiteren Menüeintrag **SPEICHERN** hinzu.

Übung 13.9 Fügen Sie Ihrer Klasse drei private Methoden **dateiOeffnen**, **dateiSpeichern** und **beenden** hinzu. Ändern Sie die Methode **actionPerformed** so ab, dass sie die entsprechende Methode aufruft, wenn ein Menüeintrag selektiert wird.

Übung 13.10 Wenn Sie **Übung 13.6** durchgeführt haben (Einfügen eines **HILFE**-Menüs), stellen Sie sicher, dass auch dieser Menüeintrag geeignet behandelt wird.

Wie wir sehen, funktioniert dieser Ansatz. Wir können nun Methoden für die einzelnen Menüeinträge implementieren, die die jeweiligen Programmaufgaben ausführen. Es gibt jedoch einen Aspekt, den wir näher untersuchen sollten: Die aktuelle Lösung ist hinsichtlich Wartbarkeit und Erweiterbarkeit nicht besonders gelungen.

Untersuchen Sie den Rumpf der Methode **actionPerformed**, die Sie für **Übung 13.9** schreiben mussten. Er birgt mehrere Probleme:

- Sie haben vermutlich eine **if**-Anweisung mit der Methode **getActionCommand** benutzt, um herauszufinden, welcher Eintrag aktiviert wurde. Beispielsweise könnten Sie schreiben:

```
if(event.getActionCommand().equals("Öffnen")) ...
```

Es ist keine gute Idee, abhängig von der Bezeichnung eines Menüeintrags eine Funktion auszuführen. Was passiert, wenn Sie Ihre Oberfläche in eine andere Sprache übersetzen? Wenn Sie den Text des Menüeintrags ändern, würde das Programm nicht mehr funktionieren. (Oder Sie müssten alle Stellen im Quelltext finden, an denen diese Zeichenkette verwendet wird, und sie entsprechend abändern – eine mühsame und fehlerträchtige Arbeit.)

- Eine zentrale „Verteiler“-Methode zur Auswahl des aufzurufenden Anwendungscodes (wie unsere Methode **actionPerformed**) ist keine gelungene Struktur. Letztendlich rufen alle Menüeinträge eine einzige Methode auf, die wiederum nichts anderes tut, als für jeden Eintrag eine zugeordnete Methode aufzurufen. Dies ist ärgerlich aus Wartungssicht (für jeden zusätzlichen Menüeintrag müssen wir eine neue **if**-Anweisung in **actionPerformed** einfügen) und wirkt auch wie überflüssiger Aufwand. Es wäre deutlich besser, wenn wir jeden Menüeintrag dazu bringen könnten, die zugeordnete Methode selbst direkt aufzurufen.

Im nächsten Abschnitt zeigen wir, wie Lambda-Ausdrücke von Java 8 den Code zur Ereignisverarbeitung deutlich vereinfachen können.

13.4.7 Lambda-Ausdrücke zur Ereignisverarbeitung

In diesem Abschnitt werden wir das Konzept der Lambda-Ausdrücke ausgiebig einsetzen, die mit Java 8 eingeführt wurden und die wir ausführlich in Kapitel 5 dieses Buchs behandelt haben.

Das Interface **ActionListener** enthält eine einzige abstrakte Methode, was bedeutet, dass es laut Definition ein *funktionales Interface* ist, wie in Kapitel 12 eingeführt. Dort hatten wir gesagt, dass ein Lambda-Ausdruck immer dann verwendet werden kann, wenn ein Objekt eines funktionalen Interface-Typs benötigt wird. Wird also die **addActionListener**-Methode eines **JMenuItem** aufgerufen wird, so können wir einen Lambda-Ausdruck benutzen, um die Aktionen festzulegen, die ausgeführt werden, wenn das entsprechende **ActionEvent** eintritt. Der folgende Code zeigt die vollständige Syntax für einen Lambda-Ausdruck, der zur Ereignisverarbeitung für den Menüeintrag **ÖFFNEN** eingesetzt wird:

```
oeffnenEintrag.addActionListener(
    (ActionEvent e) -> { dateiOeffnen(); }
);
```

Wird ein Menüeintrag ausgewählt, dann erhält der zugehörige Listener-Lambda-Ausdruck ein **ActionEvent**-Objekt – mit dem er nichts macht – und der Rumpf des Listeners ruft die Methode **dateiOeffnen** auf, die in der umschließenden **Bildbetrachter**-Klasse definiert ist. Der Rumpf eines Lambda-Ausdrucks kann auf alle Anteile der umschließenden Klasse zugreifen – einschließlich der als privat deklarierten.

Übung 13.11 Implementieren Sie die hier beschriebene Behandlung von Menüeinträgen mit inneren Klassen in Ihrer eigenen Version des **Bildbetrachters**.

Wir können die vereinfachte Syntax für Lambda-Ausdrücke benutzen, die es uns erlaubt, den Parametertyp bei Lambda-Ausdrücken mit nur einem Parameter auszulassen sowie die geschweiften Klammern bei einer einzigen Anweisung. Der Code, um den Listener hinzuzufügen, sieht dann folgendermaßen aus:

```
oeffnenEintrag.addActionListener(e -> dateiOeffnen());
```

Version 0-3 des **Bildbetrachter**-Projekts implementiert seine Listener auf diese Weise.

Übung 13.12 Öffnen Sie das Projekt **Bildbetrachter0-3** und untersuchen Sie es. Damit ist gemeint: Testen Sie es und sehen Sie sich den Quelltext an. Wundern Sie sich nicht, wenn Sie einiges nicht verstehen, denn einige neue Konstrukte werden erst in diesem Kapitel thematisiert.

Übung 13.13 Sie haben möglicherweise bemerkt, dass das Auswählen des Menüeintrags **BEENDEN** nun das Programm beendet. Sehen Sie sich an, wie dies erfolgt. Lesen Sie in der Dokumentation der beteiligten Klassen und Methoden nach.

Viele der Listener-Interfaces, die im Zusammenhang mit GUIs stehen, sind funktionale Interfaces und es ist ratsam, Listener mithilfe von Lambda-Ausdrücken zu implementieren. Wir werden dieser Konvention bei zukünftigen Versionen des *Bildbetrachter*-Projekts folgen. Für nicht funktionale Interfaces – d.h. solche, die mehr als eine abstrakte Methode haben – ist eine andere Syntax erforderlich, die wir in Abschnitt 13.8 behandeln werden: innere Klassen.

Beachten Sie auch, dass nun nicht mehr die Klasse **Bildbetrachter** den **Action-Listener** implementiert (wir haben ihre **actionPerformed**-Methode entfernt), sondern der Lambda-Ausdruck. (Lambda-Ausdrücke implementieren Interfaces allein aufgrund der Tatsache, dass sie eine Implementierung liefern, die zur Signatur der Methode im funktionalen Interface passt.) Dies erlaubt es uns, Lambda-Ausdrücke als Action-Listener für die Menüeinträge zu verwenden.

Zusammengefasst haben wir erreicht, dass nicht länger das Bildbetrachter-Objekt auf alle Ereignisse lauscht, sondern haben stattdessen für jedes mögliche Ereignis separate Listener-Objekte erzeugt – definiert durch einen Lambda-Ausdruck –, wobei jeder Listener nur mit einem Ereignistyp umgehen muss. Da jeder Listener seine eigene Implementierung der **actionPerformed**-Methode hat, können wir nun eine spezifische Behandlung in diesen Methoden angeben. Und da die Lambda-Ausdrücke im Sichtbarkeitsbereich der umschließenden Klasse liegen, können sie in der Implementierung ihres Rumpfs vollen Gebrauch von der umschließenden Klasse machen, weil sie auf die privaten Datenfelder und Methoden dieser Klasse zugreifen können.

Diese Struktur ist kohärent und erweiterbar. Wenn wir einen zusätzlichen Menüeintrag benötigen, fügen wir einfach Code zum Erzeugen des Eintrags sowie den Lambda-Ausdruck zur Behandlung der zugehörigen Funktion hinzu. Es ist keine Auflistung in einer zentralen Methode notwendig.

Der Einsatz von Lambda-Ausdrücken kann allerdings den Quelltext schwerer lesbar machen, wenn die Ausdrücke lang sind. Es wird dringend empfohlen, sie nur für sehr kurze Handler-Funktionalität und für etablierte Programmiermuster zu verwenden. Für komplexeren Code ist es besser, den Lambda-Ausdruck in einer separaten Methode aufzurufen, die in der umschließenden Klasse definiert ist – wie wir es hier getan haben – oder den Einsatz von *inneren Klassen* zu erwägen. Innere Klassen werden in Abschnitt 13.8 besprochen.

13.4.8 Zusammenfassung der zentralen GUI-Konzepte

Zu Anfang dieses Kapitels haben wir die drei Stützpfeiler der Entwicklung von GUI-Anwendungen aufgelistet: Komponenten, Layout und Ereignisverarbeitung. In den folgenden Abschnitten haben wir uns dann auf zwei dieser Themengebiete konzentriert: die Komponenten, von denen wir diverse Vertreter kennengelernt haben (Label, Knöpfe, Menüs, Menüeinträge) und die Ereignisverarbeitung, die wir in einiger Ausführlichkeit besprochen haben.

Um zum aktuellen Zustand zu gelangen – Anzeigen eines Fensters mit einem Label und einigen Menüs – mussten wir hart arbeiten, denn es gab eine Menge Hintergrundkonzepte zu diskutieren. Von jetzt an wird es einfacher – wirklich! Das Verstehen der Ereignisverarbeitung für Menüeinträge ist vermutlich das Schwierigste, das wir für unser Beispiel zu meistern hatten.

Das Hinzufügen weiterer Menüs und Komponenten zu dem Frame sollte uns jetzt keine Schwierigkeiten mehr bereiten – schließlich geht es nur darum, das Gelernte erneut anzuwenden. Der einzige neue Punkt, den wir noch ansprechen müssen, ist das *Layout* und die Frage: Wie werden Komponenten im Frame angeordnet?

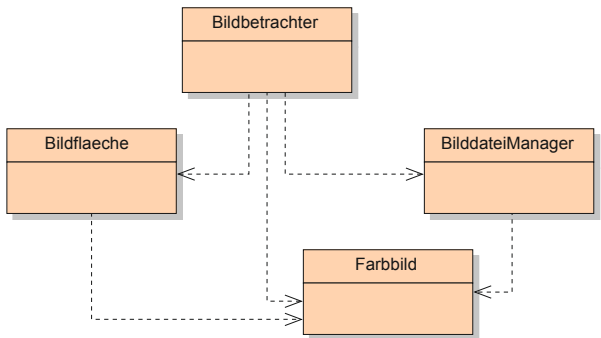
13.5 Bildbetrachter 1.0: die erste komplette Version

Wir werden nun an der ersten vollständigen Version basteln – einer Version, die wirklich ihre Hauptaufgabe erfüllt: Bilder anzeigen.

13.5.1 Klassen zur Bildverarbeitung

Auf dem Weg zur Lösung betrachten wir noch eine weitere Zwischenversion: *Bildbetrachter0-4*. Ihre Klassenstruktur ist in Abbildung 13.4 zu sehen.

Abbildung 13.4
Die Klassenstruktur
der **Bildbetrachter**-
Anwendung.



Wie Sie sehen, haben wir drei neue Klassen hinzugefügt: **Farbbild**, **Bildflaeche** und **BilddateiManager**. **Farbbild** ist eine Klasse, die Bilder repräsentiert, die wir betrachten und bearbeiten wollen. **BilddateiManager** ist eine Hilfsklasse, die uns statische Methoden liefert, mit denen Bilddateien (im JPEG- und PNG-Format) von der Festplatte gelesen und im **Farbbild**-Format geliefert werden und mit denen ein Farbbild auf der Festplatte gespeichert werden kann. **Bildflaeche** ist eine selbst geschriebene Swing-Komponente zum Anzeigen des Bildes in der GUI.

Wir werden die wichtigsten Aspekte jeder dieser Klassen etwas ausführlicher darstellen. Wir werden die Klassen jedoch nicht komplett erläutern – sie sollen dem neugierigen Leser auch noch zur eigenen Erkundung dienen können.

Die Klasse **Farbbild** ist unser selbst definiertes Format, in dem wir Bilder im Speicher halten. Sie können sich ein **Farbbild** als ein zweidimensionales Array von Bildpunkten vorstellen. Jeder Bildpunkt kann eine Farbe haben. Wir benutzen die Standardklasse **Color** (aus dem Paket **java.awt**), um die Farbe jedes Bildpunktes zu repräsentieren. (Sehen Sie sich auch die Dokumentation der Klasse **Color** an – wir werden sie später noch brauchen.)

Farbbild ist implementiert als eine Subklasse der Java-Standardklasse **BufferedImage** (aus dem Paket **java.awt.image**). Die Klasse **BufferedImage** liefert uns einen Großteil der Funktionalität, die wir benötigen (sie repräsentiert ein Bild ebenfalls als ein zweidimensionales Array), aber sie bietet keine Methoden, mit denen wir die Farbe eines Bildpunktes über ein **Color**-Objekt setzen oder auslesen können (sie benutzt intern ein anderes Format, das wir nicht benutzen wollen). Deshalb haben wir eine eigene Subklasse definiert, die diese beiden Methoden hinzufügt.

Im Rahmen dieses Projektes können wir die Klasse **Farbbild** als eine Bibliotheksklasse ansehen – Sie werden diese Klasse nicht bearbeiten müssen.

Die wichtigsten Methoden der Klasse **Farbbild** sind für uns:

- **gibPunktfarbe** und **setzePunktfarbe** zum Lesen und Verändern einzelner Bildpunkte, sowie
- **getHeight** und **getWidth** (geerbt von **BufferedImage**) zum Auslesen der Höhe und der Breite eines Bildes.

Die Klasse **BilddateiManager** hat drei Methoden: eine zum Lesen einer Bilddatei von der Festplatte, die ein **Farbbild** liefert, eine zum Schreiben eines Farbbildes auf die Festplatte und eine zum Öffnen eines Dateiauswahldialoges, damit der Benutzer interaktiv eine Bilddatei zum Laden auswählen kann. Die Methoden können Dateien in den Standardformaten JPEG und PNG lesen, die Methode zum Speichern schreibt im JPEG-Format. Dabei werden die Standard-Java-Ein-Ausgabe-Methoden für Bilder der Klasse **ImageIO** verwendet (Paket **javax.imageio**).

Die Klasse **Bildflaeche** realisiert eine selbst definierte Swing-Komponente zum Anzeigen unseres Bildes. Selbst definierte Swing-Komponenten können einfach erzeugt werden, indem eine Subklasse einer existierenden Komponente definiert wird. Auf diese Weise können sie in einen Swing-Container eingefügt werden und in unserer GUI wie jede andere Swing-Komponente angezeigt werden. **Bildflaeche** ist eine Subklasse von **JComponent**. Wichtig für uns ist vor allem, dass **Bildflaeche** eine Methode **setzeBild** anbietet, die als Parameter ein **Farbbild** zum Anzeigen bekommt.

13.5.2 Einfügen des Bildes

Nachdem wir die Klassen für den Umgang mit Bildern vorbereitet haben, ist das Einfügen des Bildes in die Benutzeroberfläche einfach. Listing 13.4 zeigt die wichtigsten Unterschiede zu den vorherigen Versionen.

Listing 13.4

Die Klasse
Bildbetrachter mit
Bildflaeche.

```
public class Bildbetrachter
{
    private JFrame fenster;
    private Bildflaeche bildflaeche;

    // Konstruktor und beenden-Methode hier ausgelassen

    /**
     * 'Datei oeffnen'-Funktion: Öffnet einen Dateiauswahldialog zur
     * Auswahl einer Bilddatei und zeigt das selektierte Bild an.
     */
    private void dateiOeffnen()
    {
        Farbbild bild = BilddateiManager.gibBild();
        bildflaeche.setzeBild(bild);
        fenster.pack();
    }

    /**
     * Erzeuge das Swing-Fenster samt Inhalt.
     */
    private void fensterErzeugen()
    {
        fenster = new JFrame("Bildbetrachter");
        menuzeileErzeugen(fenster);

        Container contentPane = fenster.getContentPane();

        bildflaeche = new Bildflaeche();
        contentPane.add(bildflaeche);

        // Aufbau abgeschlossen - Komponenten arrangieren lassen
        fenster.pack();
        fenster.setVisible(true);
    }

    // Methode menuzeileErzeugen hier ausgelassen
}
```

Wenn wir diesen Quelltext mit den vorherigen Versionen vergleichen, stellen wir nur zwei kleine Unterschiede fest:

- In der Methode **fensterErzeugen** erzeugen wir nun eine **Bildflaeche** statt einem **JLabel** und fügen sie mit **add** als Komponente ein. Das ist nicht komplizierter als mit einem Label. Das **Bildflaeche**-Objekt wird in einem Datenfeld gespeichert, damit wir später wieder darauf zugreifen können.
- Unsere Methode **dateiOeffnen** wurde so geändert, dass sie nun tatsächlich eine Bilddatei öffnet und anzeigt. Mithilfe unserer Bildverarbeitungsclassen ist dies nun auch einfach. Die Klasse **BilddateiManager** bietet eine Methode zum Selektieren und Öffnen eines Bildes und die Klasse **Bildflaeche** bietet eine Methode, um dieses Bild anzuzeigen. Beachtenswert ist, dass wir **fenster.pack()** am Ende der Methode **dateiOeffnen** aufrufen müssen, da sich die Größe unserer Bild-Komponente geändert hat. Die Methode **pack** wird das Layout des Fensters neu berechnen und anzeigen, sodass die Größenänderung korrekt behandelt wird.

Übung 13.14 Öffnen und testen Sie das Projekt *Bildbetrachter0-4*. Der Ordner für dieses Kapitel enthält auch einen Ordner namens *bilder*. Darin können Sie einige Testbilder finden, die Sie benutzen können. Natürlich können Sie auch eigene Bilder verwenden.

Übung 13.15 Was passiert, wenn Sie ein Bild öffnen und dann das Fenster in der Größe verändern? Was passiert, wenn Sie die Fenstergröße zuerst ändern und dann ein Bild öffnen?

Mit dieser Version haben wir die zentrale Aufgabe gelöst: Wir können nun eine Bilddatei aus dem Dateisystem laden und auf dem Bildschirm anzeigen. Bevor wir unser Projekt wirklich „Version 1.0“ nennen können und es damit erstmalig für vollständig erklären, wollen wir aber noch einige Verbesserungen vornehmen (siehe Abbildung 13.2):

- Wir wollen zwei Labels hinzufügen: eines zum Anzeigen des Dateinamens im oberen Bereich und eines für Statusmeldungen im unteren Bereich.
- Wir wollen ein FILTER-Menü mit einigen Filtern hinzufügen, mit denen das Aussehen des Bildes verändert werden kann.
- Wir wollen ein HILFE-Menü hinzufügen, das einen Eintrag INFO... enthält. Die Auswahl dieses Eintrags soll einen Dialog mit Informationen wie dem Namen der Anwendung und ihrer Versionsnummer anzeigen.

13.5.3 Layout

Zuerst werden wir die beiden Text-Label in unsere Oberfläche einfügen: eines oben, auf dem der Dateiname der aktuell angezeigten Bilddatei steht, und eines unten, das für verschiedene Statusmeldungen benutzt wird.

Das Erzeugen dieser Labels ist einfach – sie sind beide simple Instanzen der Klasse `JLabel`. Wir halten sie in Datenfeldern, damit wir später zum Ändern der angezeigten Texte auf sie zugreifen können. Die einzige Frage ist, wie wir sie auf dem Bildschirm arrangieren wollen.

Ein erster – naiver und inkorrekt – Ansatz könnte so aussehen:

```
Container contentPane = fenster.getContentPane();
```

```
dateinameLabel = new JLabel();
contentPane.add(dateinameLabel);
```

```
bildflaeche = new Bildflaeche();
contentPane.add(bildflaeche);
```

```
statusLabel = new JLabel("Version 1.0");
contentPane.add(statusLabel);
```

Die Idee hinter diesem Ansatz ist einfach: Wir lassen uns vom Frame die Inhaltsfläche geben und fügen, eine nach der anderen, alle Komponenten ein, die wir angezeigt bekommen wollen. Das Problem dabei ist: Wir haben nicht genau angegeben, wie die drei Komponenten arrangiert werden sollen. Wir könnten sie nebeneinander haben wollen oder übereinander oder in einem beliebigen anderen Layout. Da wir kein Layout angegeben haben, benutzt der Container (die Inhaltsfläche) ein voreingestelltes Layout. Und das, so stellt sich heraus, ist nicht das, was wir wollen.

Swing benutzt *Layout-Manager* zur Gestaltung des Layouts der Komponenten in einer GUI. Jeder Container mit enthaltenen Komponenten, die Inhaltsfläche, verfügt über einen Layout-Manager, der sich um die Ausrichtung der Komponenten im Container kümmert.

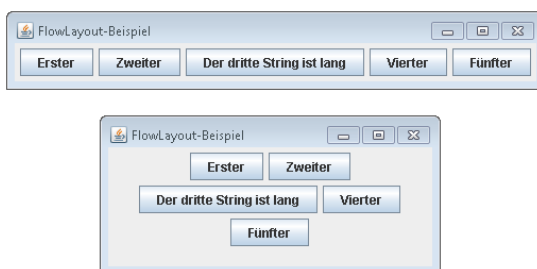
Übung 13.16 Benutzen Sie, ausgehend von Ihrer letzten Version des Projekts, das obige Quelltextfragment zum Einfügen der beiden Label. Was beobachten Sie?

Swing bietet mehrere verschiedene Layout-Manager, um verschiedene Layout-Einstellungen zu unterstützen. Die wichtigsten sind: **FlowLayout**, **BorderLayout**, **GridLayout** und **BoxLayout**. Jedes dieser Layouts wird durch eine Java-Klasse der Swing-Bibliothek repräsentiert und arrangiert die Komponenten unter seiner Kontrolle auf andere Weise.

Im Folgenden ist jedes Layout kurz beschrieben. Die Hauptunterschiede bestehen in der Art, wie die Komponenten positioniert werden und der verfügbare Bereich zwischen den Komponenten verteilt wird. Sie finden die hier besprochenen Beispiele in dem Projekt *Layouts*.

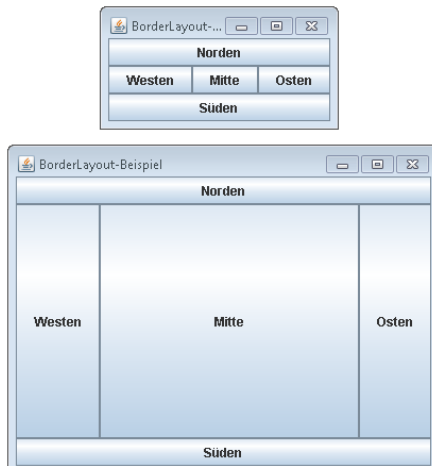
Ein **FlowLayout** (Abbildung 13.5) arrangiert alle Komponenten sequenziell von links nach rechts. Es lässt allen Komponenten ihre bevorzugte Größe und zentriert sie an der vertikalen Achse. Wenn der Platz in der Horizontalen nicht groß genug für alle Komponenten ist, werden die Komponenten auf eine weitere Zeile umgebrochen. Bei einem **FlowLayout** kann die Ausrichtung der Komponenten auch links- oder rechtsbündig gesetzt werden. Da die Komponenten nach einer Vergrößerung des Fensters den verfügbaren Raum nicht durch Anpassung ihrer Größen voll ausnutzen, gibt es leeren Raum um die Komponenten.

Abbildung 13.5
FlowLayout.



Ein **BorderLayout** (Abbildung 13.6) platziert bis zu fünf Komponenten in einem festgelegten Muster: eine in der Mitte und jeweils eine oben, unten, rechts und links. Jede dieser Positionen darf auch leer sein, sodass auch weniger als fünf Komponenten zulässig sind. Die fünf Positionen sind mit den Konstanten **CENTER**, **NORTH**, **SOUTH**, **EAST** und **WEST** der Klasse **BorderLayout** benannt. Beim **BorderLayout** wird auch nach einer Vergrößerung des Fensters der ganze Bereich von den Komponenten (ungleichmäßig verteilt) eingenommen.

Abbildung 13.6
BorderLayout.

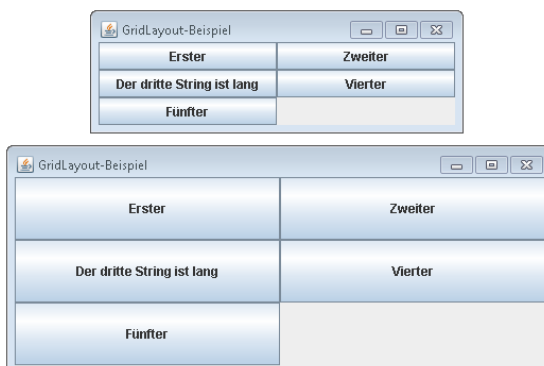


Dieses Layout mag auf den ersten Blick sehr speziell erscheinen – man fragt sich, wie häufig es verwendet wird. In der Praxis ist dies jedoch ein überraschend nützliches Layout, das in vielen Anwendungen eingesetzt wird. In BlueJ wird beispielsweise sowohl für das Hauptfenster als auch für den Editor ein **BorderLayout** als primärer Layout-Manager verwendet.

Wenn ein **BorderLayout** in der Größe verändert wird, wird die mittlere Komponente in beide Dimensionen gedehnt. Die Ost- und West-Komponenten ändern ihre Höhe, bleiben jedoch in der Breite gleich. Die Nord- und Süd-Komponenten behalten ihre Höhe und nur ihre Breite ändert sich.

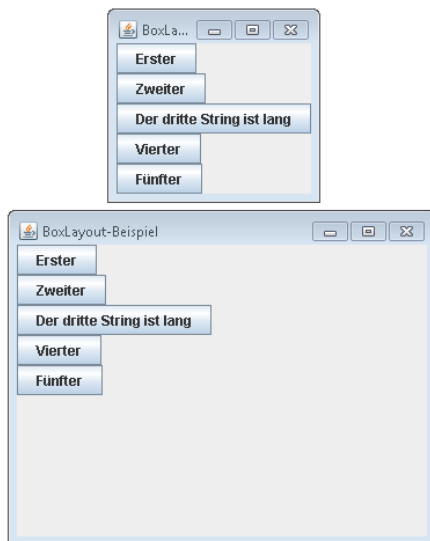
Ein **GridLayout** (Abbildung 13.7) ist nützlich, um Komponenten in einem gleichmäßigen Gitter (*grid*) zu arrangieren. Die Anzahl der Zeilen und Spalten kann angegeben werden, der **GridLayout**-Manager ordnet alle Komponenten in derselben Größe an. Dies kann nützlich sein, um beispielsweise eine Reihe von Knöpfen in die gleiche Größe zu zwingen. Die Breite einer **JButton**-Instanz ist normalerweise festgelegt durch den Text des Knopfes: Jeder Knopf wird so breit gemacht, dass der Text Platz findet. Das Einfügen von Knöpfen in ein **GridLayout** führt dazu, dass alle Knöpfe die Größe des breitesten Knopfes erhalten. Wenn eine ungerade Anzahl an gleichgroßen Komponenten ein 2-D-Gitter nicht füllen kann, kommt es in einigen Konfigurationen zu einem leeren Bereich.

Abbildung 13.7
GridLayout.



Ein **BoxLayout** arrangiert mehrere Komponenten entweder vertikal oder horizontal. Bei einer Größenänderung wird nicht umgebrochen (Abbildung 13.8). Verschachtelt man mehrere **BoxLayouts** ineinander, so können anspruchsvolle zweidimensionale Layouts erstellt werden.

Abbildung 13.8
BoxLayout.



Übung 13.17 Nutzen Sie das Projekt *Layouts* und experimentieren Sie mit den hier beschriebenen Beispielen. Fügen Sie Komponenten hinzu und entfernen Sie Komponenten aus den bestehenden Klassen, um ein Gefühl für die Hauptmerkmale der verschiedenen Layout-Stile zu bekommen. Was passiert, wenn es beispielsweise keine **CENTER**-Komponente beim **BorderLayout** gibt?

13.5.4 Geschachtelte Container

Alle oben beschriebenen Layout-Strategien sind relativ einfach. Der Schlüssel zum Bau gut aussehender und gut benutzbarer Oberflächen liegt in einem letzten Detail: Layouts können ineinander verschachtelt werden. Viele der Swing-Komponenten sind *Container*. Container erscheinen nach außen wie einzelne Komponenten, können aber viele andere Komponenten enthalten. Jeder Container verfügt über einen eigenen Layout-Manager.

Der am häufigsten verwendete Container ist die Klasse **JPanel**. Ein **JPanel** kann als Komponente in die Inhaltsfläche eines Frames eingefügt werden und in dem **JPanel** können dann mehrere Komponenten ausgerichtet werden. Abbildung 13.9 beispielsweise zeigt eine Oberflächenanordnung, die ähnlich zu der des Hauptfensters von BlueJ ist. Die Inhaltsfläche des Frames benutzt ein **BorderLayout**, bei dem die **EAST**-Position unbenutzt ist. Der **NORTH**-Bereich des **BorderLayout** enthält einen **JPanel** mit einem horizontalen **FlowLayout**, das seine Komponenten (möglicherweise Knöpfe einer Werkzeugleiste) in einer Reihe arrangiert. Die **SOUTH**-Komponente ist ähnlich: ein weiterer **JPanel** mit einem **FlowLayout**.

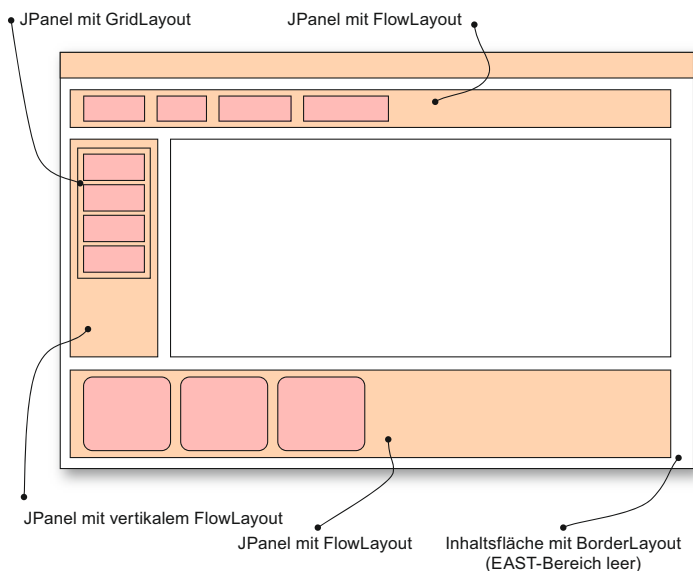


Abbildung 13.9
Erstellen einer GUI
mit geschachtelten
Containern.

Die Gruppe von Knöpfen im **WEST**-Bereich wurde zuerst in einen **JPanel** mit einem einspaltigen **GridLayout** eingefügt, um für alle Knöpfe die gleiche Breite zu erreichen. Dieser **JPanel** wurde dann in einen weiteren **JPanel** mit einem **FlowLayout** eingefügt, damit das Gitter sich nicht über die volle Höhe des **WEST**-Bereichs erstreckt. Dieser äußere **JPanel** schließlich wurde in den **WEST**-Bereich des Frames eingefügt.

Beachten Sie, wie der Container und der Layout-Manager beim Layout der Komponenten kooperieren. Der Container hält die Komponenten, aber der Layout-Manager entscheidet über ihre Anordnung auf dem Bildschirm. Jeder Container hat einen Layout-Manager. Er verwendet einen voreingestellten Layout-Manager, wenn wir nicht explizit einen setzen. Die Voreinstellung ist für verschiedene Container unterschiedlich: Die Inhaltsfläche eines **JFrame** beispielsweise hat per Voreinstellung ein **BorderLayout**, ein **JPanel** hingegen hat ein **FlowLayout** voreingestellt.

Übung 13.18 Sehen Sie sich die grafische Benutzeroberfläche des *Rechner*-Projekts aus Kapitel 9 an (Abbildung 9.6). Welche Art von Containern-/Layout-Managern wurde bei ihrer Erzeugung verwendet? Nachdem Sie diese Frage schriftlich beantwortet haben, öffnen Sie das Projekt *Rechner-GUI* und überprüfen Sie Ihre Antwort durch Lesen des Quelltextes.

Übung 13.19 Welche Layout-Manager könnten beim Layout des Editorfensters von BlueJ zum Einsatz gekommen sein?

Übung 13.20 Rufen Sie in BlueJ die Funktion **KLASSE AUS BIBLIOTHEK VERWENDEN...** aus dem **WERKZEUGE**-Menü auf. Sehen Sie sich den Dialog auf dem Bildschirm an. Welche Container-/Layout-Manager wurden hier vermutlich eingesetzt? Verändern Sie den Dialog in seiner Größe und beobachten Sie das Verhalten bei der Größenänderung, um zusätzliche Informationen zu bekommen.

Nun sollten wir wieder den Quelltext unserer *Bildbetrachter*-Anwendung untersuchen. Unser Ziel ist sehr simpel. Wir wollen drei Komponenten übereinander sehen: ganz oben ein Label, das Bild in der Mitte und unten ein weiteres Label. Dies können mehrere Layout-Manager. Welchen wir wählen sollten, wird klarer, wenn wir über das Verhalten bei Größenänderungen nachdenken. Wenn wir das Fenster vergrößern, dann sollen die Labels ihre Höhe beibehalten und das Bild soll den gesamten Zusatzplatz zugeteilt bekommen. Dies legt ein **BorderLayout** nahe: Die Labels können im **NORTH**- und **SOUTH**-Bereich liegen und das Bild im **CENTER**. Listing 13.5 zeigt den Quelltext, in dem dies umgesetzt ist.

Listing 13.5

Die Verwendung von **BorderLayout** zum Arrangieren von Komponenten.

```
private void fensterErzeugen()
{
    fenster = new JFrame("Bildbetrachter");
    menuezeileErzeugen(fenster);

    ...

    Container contentPane = fenster.getContentPane();

    contentPane.setLayout(new BorderLayout(6, 6));

    dateinameLabel = new JLabel();
    contentPane.add(dateinameLabel, BorderLayout.NORTH);

    bildflaeche = new Bildflaeche();
    contentPane.add(bildflaeche, BorderLayout.CENTER);

    statusLabel = new JLabel("Version 1.0");
    contentPane.add(statusLabel, BorderLayout.SOUTH);

    ...
}
```

Zwei Details sind hier noch erwähnenswert. Erstens: Die Methode **setLayout** wird an der Inhaltsfläche aufgerufen, um den gewünschten Layout-Manager zu setzen.⁴ Der Layout-Manager selbst ist ein Objekt, also erzeugen wir eine Instanz der Klasse **BorderLayout** und übergeben sie an die Methode **setLayout**.

Zweitens: Wenn wir eine Komponente in einen Container mit **BorderLayout** einfügen, dann verwenden wir eine andere **add**-Methode, die einen zweiten Parameter hat. Der Wert für den zweiten Parameter ist eine der öffentlichen Konstanten **NORTH**, **SOUTH**, **EAST**, **WEST** oder **CENTER**, die in der Klasse **BorderLayout** definiert sind.

Übung 13.21 Implementieren und testen Sie den oben gezeigten Quelltext in Ihrer Version des Projektes.

Übung 13.22 Experimentieren Sie mit anderen Layout-Managern. Probieren Sie alle oben genannten Layout-Manager in Ihrem Projekt aus und testen Sie, ob diese sich wie erwartet verhalten.

⁴ Um genau zu sein: Der Aufruf von **setLayout** ist an dieser Stelle nicht notwendig, da der Layout-Manager der Inhaltsfläche bereits ein **BorderLayout** ist. Wir haben den Aufruf hier zur Klarheit und Lesbarkeit eingefügt.

13.5.5 Bildfilter

Zwei Dinge bleiben noch zu tun, bis wir unsere erste Version des Bildbetrachters fertig gestellt haben: das Hinzunehmen einiger Bildfilter und das Einfügen eines HILFE-Menüs. Als Nächstes werden wir uns um die Filter kümmern.

Die Bildfilter sind der erste Schritt zur Bildmanipulation. Letztendlich wollen wir nicht nur Bilder laden und betrachten können, sondern sie auch manipulieren und geändert abspeichern können.

Wir beginnen hier mit drei einfachen Filtern. Ein Filter ist eine Funktion, die auf das gesamte Bild angewendet wird. (Er könnte auch so geändert werden, dass er nur auf Teile eines Bildes angewendet wird, aber das wollen wir vorläufig noch nicht tun.)

Die drei Filter nennen wir *Dunkler*, *Heller* und *Schwellwert*. *Dunkler* macht das ganze Bild etwas dunkler, *Heller* hellt es etwas auf. Der *Schwellwert*-Filter wandelt das Bild in ein Graustufenbild mit nur wenigen voreingestellten Graustufen um. Wir haben einen dreistufigen Schwellwert gewählt. Dies bedeutet, dass wir drei Farben verwenden: Schwarz, Weiß und ein mittleres Grau. Alle Bildpunkte, die in ihrer Helligkeit im oberen Drittel liegen, werden in weiße Punkte umgewandelt, alle im unteren Drittel in schwarze und die im mittleren Drittel in graue.

Um dies zu erreichen, müssen wir zwei Dinge tun:

- Wir müssen einen Menüeintrag samt Menü-Listener für jeden Filter erzeugen und
- wir müssen die eigentlichen Filteroperationen implementieren.

Zuerst die Menüs. Das ist nichts wirklich Neues. Es ist lediglich mehr Code in der Art, wie wir ihn schon für die existierenden Menüs geschrieben haben. Wir müssen Folgendes erledigen:

- Wir erzeugen ein neues Menü (Klasse **JMenu**) namens **FILTER** und fügen es in die Menüzeile ein.
- Wir erzeugen drei Menüeinträge (Klasse **JMenuItem**) namens **HELLER**, **DUNKLER** und **SCHWELLWERT** und fügen sie in unser **FILTER**-Menü ein.
- Zu jedem Menüeintrag fügen wir mithilfe von Lambda-Ausdrücken, wie wir sie für die anderen Menüeinträge verwendet haben, einen Action-Listener hinzu. Die Action-Listener sollten die Methoden **dunkler**, **heller** und **schwellwert** entsprechend aufrufen.

Nachdem wir das Menü eingefügt und die (anfangs leeren) Methoden implementiert haben, müssen wir nun die Filter implementieren.

Die einfachste Art von Filter besteht aus einer Iteration über alle Bildpunkte eines Bildes, bei der an jedem Bildpunkt eine Änderung der Farbe vorgenommen wird. Ein Muster für diesen Prozess zeigt Listing 13.6. Kompliziertere Filter könnten auch die Werte benachbarter Bildpunkte mit einbeziehen, um den neuen Farbwert eines Bildpunktes zu berechnen.

Übung 13.23 Fügen Sie das neue Menü und die Menüeinträge in Ihre Version des Projekts ein wie hier beschrieben. Um die Action-Listener eintragen zu können, müssen Sie die drei Methoden **dunkler**, **heller** und **schwellwert** als private Methoden in Ihre **Bildbetrachter**-Klasse einfügen. Sie alle haben den Rückgabewert **void** und übernehmen keine Parameter. Diese Methoden können anfangs leere Rümpfe haben oder sie könnten einfach ausgeben, dass sie aufgerufen worden sind.

Listing 13.6
Muster für einen
einfachen Filter-
prozess.

```
for(int y = 0; y < hoehe; y++) {
    for(int x = 0; x < breite; x++) {
        Color farbe = gibPunktfarbe(x, y);

        // Ändern des Farbwerts

        setzePunktfarbe(x, y, farbe);
    }
}
```

Die Filterfunktion operiert auf dem Bild und sollte deshalb laut Entwurf nach Zuständigkeiten in der Klasse **Farbbild** implementiert sein. Andererseits schließt die Behandlung eines Menüaufrufs auch GUI-bezogenen Code ein (beispielsweise müssen wir bei einem Filteraufruf prüfen, ob überhaupt ein Bild offen ist), der eher in die Klasse **Bildbetrachter** gehört.

Aufgrund dieser Überlegungen erzeugen wir zwei Methoden, eine in **Bildbetrachter** und eine in **Farbbild**, um die Arbeit aufzuteilen (Listing 13.7 und Listing 13.8). Wir können sehen, dass die Methode **dunkler** in **Bildbetrachter** den Teil der Aufgabe enthält, der sich auf die GUI bezieht (Prüfen, ob ein Bild geladen ist, Anzeigen einer Statusmeldung, Neuanzeigen des Frames), während die Methode **abdunkeln** in **Farbbild** die eigentliche Arbeit übernimmt, alle Bildpunkte des Bildes etwas abzudunkeln.

Listing 13.7
Die Filtermethode
in der Klasse
Bildbetrachter.

```
public class Bildbetrachter
{
    // Datenfelder, Konstruktor und alle anderen Methoden ausgelassen

    /**
     * 'Dunkler'-Funktion: Das aktuelle Bild etwas abdunkeln.
     */
    private void dunkler()
    {
        if(aktuellesBild != null) {
            aktuellesBild.abdunkeln();
            fenster.repaint();
            statusAnzeigen("Angewendet: Dunkler");
        }
        else {
            statusAnzeigen("Kein Bild geladen.");
        }
    }
}
```

```

public class Farbbild extends BufferedImage
{
    // Datenfelder, Konstruktor und alle anderen Methoden ausgelassen

    /**
     * Dunkle diese Bild etwas ab.
     */
    public void abdunkeln()
    {
        int hoehe = getHeight();
        int breite = getWidth();

        // Auf alle Bildpunkte die Operation "darker" der
        // Klasse Color anwenden.
        for(int y = 0; y < hoehe; y++) {
            for(int x = 0; x < breite; x++) {
                setzePunktfarbe(x, y, gibPunktfarbe(x, y).darker());
            }
        }
    }
}

```

Listing 13.8

Implementierung eines Filters in der Klasse **Farbbild**.

Übung 13.24 Was bewirkt der Methodenaufruf **fenster.repaint()**, den Sie in der Methode **dunkler** sehen?

Übung 13.25 Wir können den Aufruf einer Methode **statusAnzeigen** sehen, der offensichtlich ein interner Methodenaufruf ist. Aufgrund des Namens können wir erraten, dass diese Methode eine Statusmeldung mithilfe des bereits besprochenen Status-Labels ausgeben soll. Implementieren Sie diese Methode in Ihrer Version des Projekts *Bildbetrachter0-4*. (*Hinweis*: Sehen Sie sich die Methode **setText** in der Klasse **JLabel** an.)

Übung 13.26 Was passiert, wenn der Menüeintrag **DUNKLER** selektiert wird, aber kein Bild geladen ist?

Übung 13.27 Erklären Sie detailliert, wie die Methode **abdunkeln** in der Klasse **Farbbild** funktioniert. (*Hinweis*: Sie enthält einen Methodenaufruf einer Methode **darker**. Zu welcher Klasse gehört diese zweite Methode? Schlagen Sie es nach.)

Übung 13.28 Implementieren Sie den Filter zum Aufhellen in der Klasse **Farbbild**.

Übung 13.29 Implementieren Sie den Schwellwert-Filter. Um die Helligkeit eines Bildpunktes zu bekommen, können Sie seinen Rot-, Grün- und Blauwert abfragen und diese aufaddieren. Die Klasse **Color** definiert statische Referenzen zu passenden Schwarz-, Weiß- und Grau-Objekten.

Sie finden eine lauffähige Implementierung aller bisher beschriebenen Aspekte im Projekt *Bildbetrachter1-0*. Sie sollten jedoch die Übungen selbst durchführen, bevor Sie sich die Lösung ansehen.

13.5.6 Dialoge

Unsere letzte Aufgabe für diese Version ist das Hinzufügen eines HILFE-Menüs mit einem Menüeintrag INFO... . Wenn dieser Eintrag selektiert wird, dann soll ein Dialogfenster erscheinen, das eine kurze Information anzeigt.

Nun müssen wir die Methode **zeigeInfo** implementieren, sodass diese den Informationsdialog anzeigt.

Übung 13.30 Fügen Sie ein weiteres Menü HILFE hinzu. Fügen Sie in dieses einen Menüeintrag mit der Beschriftung INFO... ein.

Übung 13.31 Fügen Sie eine leere Methode (eine Methode mit einem leeren Rumpf) namens **zeigeInfo** ein und tragen Sie bei dem neuen Menüeintrag eine Ereignisverarbeitung ein, die **zeigeInfo** aufruft.

Eine der wichtigsten Eigenschaften eines Dialogs ist, ob er *modal* ist oder nicht. Ein modaler Dialog blockiert alle Interaktionen mit der Anwendung, bis der Dialog wieder geschlossen wird. Er zwingt den Benutzer, auf den Dialog zu reagieren. Nichtmodale Dialoge hingegen erlauben Interaktionen in anderen Frames während der Dialog sichtbar ist.

Dialoge können auf ähnliche Weise implementiert werden wie der **JFrame** des Hauptfensters. Sie benutzen häufig die Klasse **JDialog** zum Anzeigen ihres Fensters.

Für modale Dialoge mit einer Standardstruktur gibt es allerdings einige bequeme Methoden in der Klasse **JOptionPane**, die das Anzeigen solcher Dialoge stark vereinfachen. **JOptionPane** verfügt, neben anderen Dingen, über statische Methoden, um drei Typen von Standarddialogen anzuzeigen. Diese sind:

- **Meldungsdialo**g (*message dialog*): Dies ist ein Dialog, der eine Meldung anzeigt und einen OK-Knopf zum Schließen des Dialogs hat.
- **Auswahldialo**g (*confirm dialog*): Dieser Dialog zeigt üblicherweise eine Frage an und bietet dem Benutzer einige Knöpfe zur Auswahl einer Antwort, beispielsweise JA, NEIN und ABBRECHEN.
- **Eingabedialo**g (*input dialog*): Dieser Dialog enthält eine Eingabeaufforderung und ein Textfeld, in das der Benutzer Text eingeben kann.

Unsere Info-Box ist ein einfacher Meldungsdialog. Wenn wir uns die Dokumentation der Klasse **JOptionPane** ansehen, finden wir statische Methoden mit dem Namen **showMessageDialog** für diesen Zweck.

Übung 13.32 Finden Sie die Dokumentation zu **showMessageDialog**. Wie viele Methoden mit diesem Namen gibt es? Worin unterscheiden sich diese? Welche sollten wir für unsere Info-Box benutzen? Warum?

Übung 13.33 Implementieren Sie die Methode **zeigeInfo** in Ihrer **Bildbetrachter**-Klasse, indem Sie den Aufruf einer **showMessageDialog**-Methode verwenden.

Übung 13.34 Die `showInputDialog`-Methoden der Klasse `JOptionPane` erlauben es, den Benutzer bei Bedarf zu einer Eingabe aufzufordern. Die Komponente `JTextField` hingegen ermöglicht die ständige Anzeige eines Texteingabefelds in einer GUI. Finden Sie die Dokumentation dieser Klasse. Welche Eingabe bewirkt, dass ein mit einem `JTextField` verknüpfter Action-Listener benachrichtigt wird? Kann ein Benutzer am Bearbeiten des Textes im Textfeld gehindert werden? Ist es möglich, dass ein Listener über beliebige Änderungen am Text informiert wird? (*Hinweis: In welcher Weise verwendet ein `JTextField` ein `Document`-Objekt?*)

Ein `JTextField` wird beispielsweise im Projekt *Rechner* in Kapitel 9 verwendet.

Nachdem wir uns die Dokumentation angesehen haben, können wir nun unsere Info-Box implementieren, indem wir einen Aufruf der Methode `showMessageDialog` einfügen. Der Quelltext ist in Listing 13.9 zu sehen. Beachten Sie, dass wir eine String-Konstante `VERSION` eingeführt haben, die die aktuelle Versionsnummer hält.

```
/**
 * 'Info'-Funktion: Zeige Informationen zur Anwendung.
 */
private void zeigeInfo()
{
    JOptionPane.showMessageDialog(fenster,
        "Bildbetrachter\n" + VERSION,
        "Info zu Bildbetrachter",
        JOptionPane.INFORMATION_MESSAGE);
}
```

Listing 13.9
Anzeigen eines
modalen Dialogs.

Dies war die letzte Aufgabe zur Vervollständigung von „Version 1.0“ unserer **Bildbetrachter**-Anwendung. Wenn Sie alle Übungen durchgeführt haben, dann haben Sie jetzt eine Version des Projektes, mit der Sie einiges tun können: Bilder öffnen und anzeigen, Filter anwenden, Statusmeldungen ausgeben und einen Dialog anzeigen.

Das Projekt *Bildbetrachter1-0*, das Sie im Zusatzmaterial zum Buch finden, enthält eine Implementierung der bisher behandelten Funktionalität. Sie sollten sich diese Lösung sorgfältig ansehen und sie mit Ihrer eigenen vergleichen.

In dieser Version haben wir auch die Methode `dateiOeffnen` überarbeitet, um eine bessere Benachrichtigung im Fehlerfall zu erreichen. Wenn der Benutzer eine Datei auswählt, die kein gültiges Bildformat hat, können wir nun eine vernünftige Fehlermeldung anzeigen. Nachdem wir jetzt über Meldungsdialoge Bescheid wissen, ist das eine einfache Sache.

13.5.7 Zusammenfassung der Layout-Verwaltung

In diesem Abschnitt haben wir einige spezielle Klassen für den Umgang mit Bildern geschrieben. Wichtiger noch für unsere GUI war aber, dass wir uns eingehender mit dem Layout der Komponenten beschäftigt haben und dabei sehen konnten, wie Container und Layout-Manager zusammenarbeiten, um auf dem Bildschirm exakt die Komponentenanzuordnung anzuzeigen, die wir wünschen.

Wie man Layout-Manager einsetzt, lernt man vor allem, indem man Erfahrungen sammelt und hin und wieder Layouts nach dem Versuch-und-Irrtum-Prinzip ein-

fach ausprobiert. Mit der Zeit werden Ihnen die Layout-Manager aber immer vertrauter werden.

Damit hätten wir die Grundlagen aller wichtigen Bereiche der GUI-Programmierung behandelt. Im Rest des Kapitels können wir uns daher darauf konzentrieren, unsere Anwendung noch weiter zu verbessern und ihr den letzten Schliff zu geben.

13.6 Bildbetrachter 2.0: die Programmstruktur verbessern

Version 1.0 unserer Anwendung hat eine benutzbare GUI und kann Bilder anzeigen. Sie bietet außerdem drei Filter an.

Der nächste konsequente Schritt zur Verbesserung unserer Anwendung ist das Hinzufügen weiterer interessanter Filter. Bevor wir uns darauf stürzen, sollten wir uns jedoch gut überlegen, was das für uns bedeutet.

Mit der aktuell realisierten Struktur für Filter müssen wir für jeden Filter drei Dinge tun:

- 1 einen Menüeintrag hinzufügen,
- 2 eine Methode zum Verarbeiten der Aktivierung des Menüeintrags hinzufügen und
- 3 eine Implementierung des Filters in **Farbbild** hinzufügen.

Die Punkte 1 und 3 sind unvermeidlich – wir brauchen einen Menüeintrag und eine Filterimplementierung. Aber Punkt 2 sieht verdächtig aus. Wenn wir uns diese Methoden in der Klasse **Bildbetrachter** ansehen (Listing 13.10 zeigt zwei von ihnen als Beispiel), dann sieht das nach reichlicher Code-Duplizierung aus. Diese Methoden machen alle im Prinzip das Gleiche (abgesehen von einigen Details) und – was noch schlimmer ist – wir müssen für jeden neuen Filter eine weitere dieser fast gleichen Methoden hinzufügen.

Wir wissen, dass Code-Duplizierung ein Zeichen für einen schlechten Entwurf ist und vermieden werden sollte. Wir reagieren darauf, indem wir unseren Quelltext restrukturieren. Wir suchen nach einem Entwurf, durch den wir nicht für jeden neuen Filter eine Methode zum Weiterleiten implementieren müssen.

Listing 13.10

Zwei der Methoden zur Filter-Behandlung in **Bildbetrachter**.

```
private void heller()
{
    if(aktuellesBild != null) {
        aktuellesBild.aufhellen();
        fenster.repaint();
        statusAnzeigen("Angewendet: Heller");
    }
    else {
        statusAnzeigen("Kein Bild geladen.");
    }
}

private void schwellwert()
{
    if(aktuellesBild != null) {
        aktuellesBild.schwellwerteAnwenden();
        fenster.repaint();
        statusAnzeigen("Angewendet: Schwellwert");
    }
    else {
        statusAnzeigen("Kein Bild geladen.");
    }
}
```

Um dies zu erreichen, sollten wir vermeiden, dass alle Filter in der Klasse **Bildbetrachter** „hart verdrahtet“ sind. Stattdessen verwenden wir eine Sammlung von Filtern und schreiben eine einzelne Aktivierungsmethode, die den richtigen Filter findet und aufruft. Dies entspricht der Einführung einer **agieren**-Methode beim Entkoppeln des Simulators von den einzelnen Akteur-Typen im Projekt *Fuechse-und-Hasen* (Kapitel 12).

Damit dies möglich wird, müssen zuerst die Filter selbst zu Objekten werden und nicht nur einfache Methodennamen. Wenn wir sie in einer gemeinsamen Sammlung ablegen und aus dieser Sammlung aufrufen wollen, dann benötigen alle Filter eine gemeinsame Superklasse, die wir **Filter** nennen wollen und die die Methode **anwenden** enthalten soll (Abbildung 13.10 zeigt die Struktur und Listing 13.11 den Quelltext).

Jeder Filter hat einen Namen und eine Methode **anwenden**, die den Filter auf ein Bild anwendet. Beachten Sie, dass diese Klasse abstrakt ist, da die Methode **anwenden** auf dieser Ebene abstrakt sein muss, aber die Methode **gibName** kann vollständig implementiert werden, sodass es kein Interface ist.

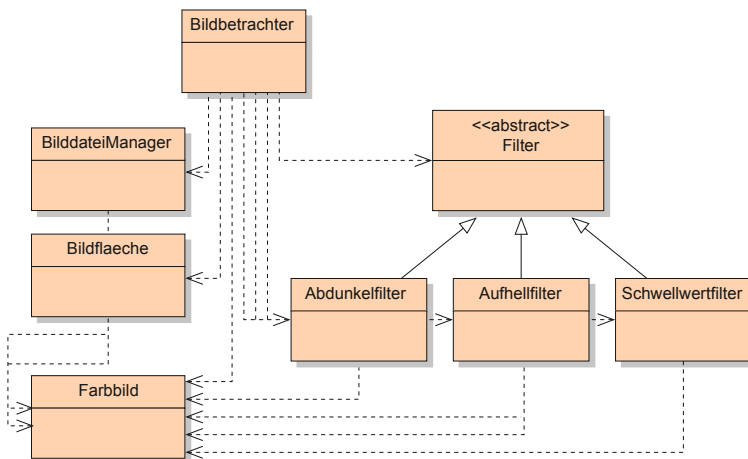


Abbildung 13.10
Klassenstruktur für
Filter-Objekte.

```

public abstract class Filter
{
    private String name;

    /**
     * Erzeuge einen neuen Filter mit dem angegebenen Namen.
     * @param name der Name des Filters
     */
    public Filter(String name)
    {
        this.name = name;
    }

    /**
     * Liefere den Namen dieses Filters.
     * @return den Namen dieses Filters
     */
    public String gibName()
    {
        return name;
    }

    /**
     * Wende diesen Filter auf das gegebene Bild an.
     * @param bild das Bild, das dieser Filter verändern soll
     */
    public abstract void anwenden(Farbbild bild);
}

```

Listing 13.11
Die abstrakte Klasse
Filter: Superklasse
für alle Filter.

Nachdem wir die Superklasse geschrieben haben, lassen sich spezifische Filter leicht als Subklassen implementieren. Wir müssen lediglich eine Implementierung der Methode **anwenden** erstellen, die das Bild (das als Parameter übergeben wird) über die Methoden **gibPunktfarbe** und **setzePunktfarbe** manipuliert. Listing 13.12 zeigt ein Beispiel.

Listing 13.12

Die Implementierung einer spezifischen Filterklasse.

```
/**
 * Ein Bildfilter zum leichten Abdunkeln eines Bildes.
 *
 * @author Michael Kölling und David J. Barnes
 * @version 1.0
 */
public class Abdunkelfilter extends Filter
{
    /**
     * Konstruktor für Objekte der Klasse Abdunkelfilter.
     * @param name der Name des Filters
     */
    public Abdunkelfilter(String name)
    {
        super(name);
    }

    /**
     * Wende diesen Filter auf das gegebene Bild an.
     * @param bild das Bild, das dieser Filter verändern soll
     */
    public void anwenden(Farbbild bild)
    {
        int hoehe = bild.getHeight();
        int breite = bild.getWidth();

        // Auf alle Bildpunkte die Operation "darker" der
        // Klasse Color anwenden.
        for(int y = 0; y < hoehe; y++) {
            for(int x = 0; x < breite; x++) {
                bild.setzePunktfarbe(x, y, bild.gibPunktfarbe(x, y).darker());
            }
        }
    }
}
```

Als Nebeneffekt kann die Klasse **Farbbild** nun stark vereinfacht werden, da alle Filtermethoden aus ihr entfernt werden können. Sie definiert nun nur noch die Methoden **setzePunktfarbe** und **gibPunktfarbe**.

Wenn alle Filter auf diese Weise definiert wurden, können wir **Filter**-Objekte erzeugen und in einer Sammlung halten (Listing 13.13).

Listing 13.13

Eine Sammlung von Filtern anlegen.

```
public class Bildbetrachter
{
    // andere Datenfelder und Kommentare ausgelassen

    private List<Filter> filterliste;

    /**
     * Erzeuge einen Bildbetrachter und zeige seine GUI auf
     * dem Bildschirm an.
     */
    public Bildbetrachter()
    {
        filterliste = filterlisteErzeugen();
        ...
    }
}
```

```

/**
 * Erzeuge und liefere eine Liste mit allen bekannten Filtern.
 * @return die Liste der Filter
 */
private List<Filter> filterlisteErzeugen()
{
    List<Filter> filterliste = new ArrayList<>();
    filterliste.add(new Abdunkelfilter("Dunkler"));
    filterliste.add(new Aufhellfilter("Heller"));
    filterliste.add(new Schwellwertfilter("Schwellwert"));

    return filterliste;
}

// andere Methoden hier ausgelassen
}

```

Sobald wir diese Struktur umgesetzt haben, können wir die beiden letzten notwendigen Änderungen vornehmen:

- Wir ändern den Code, der die Filtermenüeinträge erzeugt, so ab, dass dieser über die Filtersammlung iteriert. Für jeden enthaltenen Filter erzeugt er einen Menüeintrag und benutzt die Filtermethode `gibName`, um den Namen des zugehörigen Labels festzulegen.
- Danach können wir eine generische Methode `filterAnwenden` schreiben, die einen Filter als Parameter bekommt und diesen auf das aktuelle Bild anwendet.

Das Projekt *Bildbetrachter2-0* enthält eine vollständige Implementierung dieser Änderungen.

Übung 13.35 Öffnen Sie das Projekt *Bildbetrachter2-0*. Sehen Sie sich die Implementierung der neuen Methode zum Erzeugen und Anwenden der Filter in der Klasse **Bildbetrachter** genau an. Achten Sie besonders auf die Methoden `menuezeileErzeugen` und `filterAnwenden`. Erläutern Sie ausführlich, wie die Menüeinträge für das Filtermenü erzeugt werden und was bei ihrer Aktivierung geschieht. Zeichnen Sie ein Objektdiagramm.

Übung 13.36 Was muss getan werden, um einen neuen Filter dem Bildbetrachter hinzuzufügen?

Übung 13.37 Zusatzaufgabe. Vielleicht ist Ihnen aufgefallen, dass die **anwenden**-Methoden von allen **Filter**-Subklassen ganz ähnlich strukturiert sind: Iteriere über das ganze Bild und ändere den Wert von jedem Bildpunkt unabhängig von den umgebenden Bildpunkten. Es sollte möglich sein, diese Duplizierung genau so zu isolieren wie bei der Erzeugung der Klasse **Filter**.

Erzeugen Sie eine Methode in der Klasse **Filter**, die über das Bild iteriert und eine filterspezifische Transformation auf jeden einzelnen Bildpunkt anwendet. Ersetzen Sie die Rümpfe der Methode **anwenden** in den drei **Filter**-Subklassen durch einen Aufruf dieser Methode, wobei Sie das Bild und etwas, das die entsprechende Transformation ausführen kann, als Parameter übergeben.

In diesem Abschnitt haben wir ausschließlich Quelltext restrukturiert. Wir haben die Funktionalität der Anwendung gar nicht verändert, sondern haben ausschließlich die Struktur der Implementierung verbessert, um zukünftige Änderungen zu erleichtern.

Nachdem wir dieses Refactoring abgeschlossen haben, sollten wir testen, ob noch alles wie gewünscht funktioniert. In allen Entwicklungsprojekten benötigen wir Phasen wie diese. Wir treffen nicht immer von Anfang an perfekte Entwurfsentscheidungen, Anwendungen wachsen und die Anforderungen ändern sich. Obwohl unser Hauptaugenmerk in diesem Kapitel der GUI-Konstruktion gilt, mussten wir einen Schritt zurücktreten und unseren Quelltext restrukturieren, bevor wir weiterarbeiten. Dies wird sich auf lange Sicht auszahlen, weil spätere Änderungen leichter fallen werden.

Manchmal ist die Versuchung groß, Strukturen unverändert zu lassen, auch wenn wir erkennen, dass sie nicht gut sind. Sich mit ein wenig Code-Duplizierung zu arrangieren mag kurzfristig einfacher sein als ein sorgfältiges Refactoring. Man kommt eine Weile damit durch, aber in lang laufenden Projekten führt es zwangsläufig irgendwann zu Problemen. Als generelle Regel gilt deshalb: Nehmen Sie sich die Zeit, Ihren Code sauber zu halten.

Nachdem wir mit dem Aufräumen fertig sind, können wir nun weitere Filter hinzufügen.

Übung 13.38 Fügen Sie dem Projekt einen *Graustufenfilter* hinzu. Der Filter wandelt das Bild in ein Schwarzweißbild mit Graustufen um. Sie können einem Bildpunkt eine beliebige Graustufe zuweisen, indem Sie allen Farbkomponenten (Rot, Grün, Blau) denselben Wert geben. Die Helligkeit jedes Bildpunktes sollte unverändert bleiben.

Übung 13.39 Fügen Sie einen *Spiegelfilter* hinzu, der das Bild an der vertikalen Achse spiegelt. Der Bildpunkt in der linken oberen Ecke wandert dabei in die rechte obere Ecke und umgekehrt, sodass ein Effekt wie beim Betrachten des Bilds in einem Spiegel entsteht.

Übung 13.40 Fügen Sie einen *Negativfilter* hinzu, der jede Farbe invertiert. „Invertieren“ bei einer Bildpunkt bedeutet, seinen Farbwert x mit dem Wert $255 - x$ zu ersetzen.

Übung 13.41 Fügen Sie einen *Weichzeichnerfilter* hinzu. Ein Weichzeichnerfilter ersetzt den Farbwert jedes Bildpunkts mit dem Durchschnitt seiner benachbarten Bildpunkte und ihm selbst (insgesamt also neun Bildpunkte). Sie müssen an den Kanten des Bilds vorsichtig sein, weil dort einige Nachbarpunkte nicht existieren. Sie müssen auch auf einer Zwischenkopie des Bilds arbeiten, weil das Ergebnis sonst nicht korrekt ist. (Warum?) Sie können sich leicht eine Kopie des Bilds erzeugen, indem Sie dem Konstruktor von **Farbbild** das Original als Parameter übergeben.

Übung 13.42 Implementieren Sie einen *Solarisationsfilter*. Solarisation tritt auf, wenn ein Foto-Negativ mit einem bereits entwickelten Negativ belichtet wird. Wir können dies simulieren, indem wir die Farbkomponente jedes Bildpunkts mit einem Wert w niedriger als 128 mit $255 - w$ ersetzen. Die helleren Komponenten (mit einem Wert von 128 oder höher) lassen wir unverändert. (Dies ist ein sehr einfacher Algorithmus zur Solarisation – Sie können sehr viel bessere in der Literatur finden.)

Übung 13.43 Implementieren Sie einen *Kantenerkennungsfilter*. Dazu analysieren Sie die neun Bildpunkte in dem 3×3 -Quadrat um einen Bildpunkt (ähnlich wie beim Weichzeichnerfilter) und setzen den Wert des mittleren Bildpunkts auf die Differenz zwischen dem höchsten und dem niedrigsten Wert in der Gruppe. Tun Sie dies für jede Farbkomponente (Rot, Grün, Blau). Dies sieht insbesondere dann gut aus, wenn Sie das Bild auch invertieren.

Übung 13.44 Experimentieren Sie mit Ihren Filtern auf verschiedenen Bildern. Wenden Sie auch mehrere Filter nacheinander an.

Sobald Sie einige eigene Filter implementiert haben, sollten Sie die Versionsnummer Ihres Projekts auf „Version 2.1“ erhöhen.

13.7 Bildbetrachter 3.0: weitere GUI-Komponenten

Bevor wir das *Bildbetrachter*-Projekt abschließen, wollen wir noch einige Verbesserungen vornehmen und dabei einen Blick auf zwei weitere GUI-Komponenten werfen: Knöpfe und Rahmen.

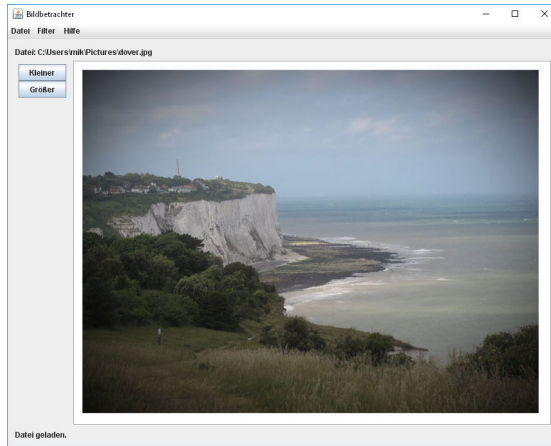
13.7.1 Knöpfe

Wir wollen den Bildbetrachter nun um die Möglichkeit erweitern, die Größe eines Bildes zu ändern. Dazu bieten wir zwei Funktionen an: **größer**, die die Größe des Bilds verdoppelt, und **kleiner**, die die Größe halbiert. (Um genau zu sein: Wir verdoppeln oder halbieren die Seitenlängen, nicht die Fläche.)

Eine Möglichkeit wäre, diese Funktionalität über Filter zu implementieren. Aber wir entscheiden uns dagegen. Bisher haben Filter nicht die Größe eines Bilds verändert, und das wollen wir beibehalten. Stattdessen führen wir eine Werkzeugleiste am linken Rand unseres Fensters ein, die zwei Knöpfe enthält, die mit GRÖßER und KLEINER beschriftet sind (Abbildung 13.11). Dies gibt uns auch die Möglichkeit, noch ein wenig mit Knöpfen, Containern und Layout-Managern zu experimentieren.

Abbildung 13.11

Bildbetrachter mit
Knöpfen in einer
Werkzeugleiste.



Bisher benutzt unser Fenster ein **BorderLayout**, bei dem der **WEST**-Bereich leer ist. Wir können diesen Bereich benutzen, um unsere Werkzeugleiste einzufügen. Es gibt nur ein kleines Problem. Der **WEST**-Bereich bei einem **BorderLayout** kann nur eine Komponente halten, wir haben aber zwei Knöpfe.

Die Lösung ist einfach. Wir fügen einen **JPanel** in den **WEST**-Bereich des Frame (wir wissen, ein **JPanel** ist ein Container) ein und platzieren die beiden Knöpfe (vom Typ **JButton**) im **JPanel**. Listing 13.14 zeigt den Quelltext dazu.

Listing 13.14

Ein **JPanel** als Werk-
zeugleiste mit zwei
Knöpfen.

```
// Die Werkzeugleiste mit den Knöpfen erzeugen
JPanel werkzeugleiste = new JPanel();

kleinerKnopf = new JButton("Kleiner");
werkzeugleiste.add(kleinerKnopf);

groesserKnopf = new JButton("Größer");
werkzeugleiste.add(groesserKnopf);

contentPane.add(werkzeugleiste, BorderLayout.WEST);
```

Übung 13.45 Fügen Sie zwei Knöpfe, beschriftet mit **GRÖßER** und **KLEINER**, in die neueste Version Ihres Projekts ein, ähnlich wie in Listing 13.14 beschrieben. Testen Sie. Was beobachten Sie?

Wenn wir dies ausprobieren, erkennen wir, dass es teilweise funktioniert, aber nicht ganz wie erwartet. Das liegt daran, dass **JPanel** per Voreinstellung ein **FlowLayout** verwendet, das seine Komponenten horizontal arrangiert. Wir möchten sie jedoch vertikal anordnen.

Wir können dies erreichen, indem wir einen anderen Layout-Manager verwenden. Ein **GridLayout** leistet das, was wir wollen. Bei der Erzeugung eines **GridLayout** legen Konstruktorparameter fest, wie viele Reihen und Spalten wir haben wollen. Ein Wert von null hat dabei eine spezielle Bedeutung, er steht für „so viele wie notwendig“.

Somit können wir ein einspaltiges **GridLayout** erzeugen, indem wir 0 als die Anzahl der Reihen angeben und 1 als die Anzahl der Spalten. Wir können dieses **GridLay-**

out dann für unser `JPanel` verwenden, indem wir die Methode `setLayout` unmittelbar nach seiner Erzeugung aufrufen.

```
JPanel werkzeugleiste = new JPanel();
werkzeugleiste.setLayout(new GridLayout(0,1));
```

Alternativ kann der Layout-Manager auch bei der Erzeugung des Containers als Konstruktorparameter angegeben werden:

```
JPanel werkzeugleiste = new JPanel(new GridLayout(0,1));
```

Übung 13.46 Ändern Sie Ihre Implementierung so ab, dass die Werkzeugleiste wie oben beschrieben ein `GridLayout` verwendet. Testen Sie. Was beobachten Sie?

Wenn wir dies ausprobieren, sehen wir schon eine Verbesserung, aber zufrieden sind wir immer noch nicht. Unsere Knöpfe sind viel größer als gewünscht. Dies liegt daran, dass ein Container in einem `BorderLayout` (der `JPanel` als Werkzeugleiste in diesem Fall) immer seinen Bereich voll ausfüllt (den `WEST`-Bereich in unserem Frame). Und ein `GridLayout` vergrößert seine Komponenten immer so weit, dass sie den kompletten Container ausfüllen.

Ein `FlowLayout` tut dies nicht – es ist ganz zufrieden, wenn es etwas Platz um seine Komponenten lassen kann. Unsere Lösung sollte deshalb beide verwenden: Ein `GridLayout` arrangiert die Knöpfe in einer Spalte und darum herum erlaubt ein `FlowLayout` leeren Platz. Wir enden also mit einem `GridLayout` innerhalb eines `FlowLayout` innerhalb eines `BorderLayout`. Listing 13.15 zeigt diese Lösung. Konstruktionen dieser Art sind sehr verbreitet. Sehr häufig werden Sie verschiedene Container ineinanderschachteln, um das richtige Erscheinungsbild zu bekommen.

```
// Die Werkzeugleiste mit den Knöpfen erzeugen
JPanel werkzeugleiste = new JPanel();
werkzeugleiste.setLayout(new GridLayout(0, 1));

kleinerKnopf = new JButton("Kleiner");
werkzeugleiste.add(kleinerKnopf);

groesserKnopf = new JButton("Größer");
werkzeugleiste.add(groesserKnopf);

// Abstände: Werkzeugleiste in ein Extra-Panel mit Flow-Layout legen
JPanel flow = new JPanel();
flow.add(werkzeugleiste);

contentPane.add(flow, BorderLayout.WEST);
```

Listing 13.15

Ein Container mit `GridLayout`, geschachtelt in einen Container mit `FlowLayout`.

Die Knöpfe sehen unserer Zielvorstellung jetzt schon sehr ähnlich. Bevor wir ihnen den letzten Schliff geben, sollten wir uns zuerst darauf konzentrieren, dass die Knöpfe auch funktionieren.

Wir brauchen zwei Methoden, beispielsweise `bildVerkleinern` und `bildVergroessern`, die die eigentliche Arbeit übernehmen, und müssen zwei Action-Listener bei den Knöpfen anmelden, die diese Methoden aufrufen.

Übung 13.47 Fügen Sie Ihrem Projekt zwei Methoden namens **bildVerkleinern** und **bildVergroessern** hinzu. Fügen Sie anfangs nur eine **println**-Anweisung in die Methodenrumpfe ein, um ihren Aufruf überprüfen zu können. Die Methoden können privat deklariert werden.

Übung 13.48 Fügen Sie den Knöpfen Lambda-Ausdrücke zur Ereignisverarbeitung hinzu, die die beiden neuen Methoden aufrufen. Das Einfügen von Ereignisverarbeitung bei Knöpfen ist identisch zum Einfügen bei Menüeinträgen. Sie können einfach das Programmiermuster von dort kopieren. Testen Sie. Stellen Sie sicher, dass die Methoden **bildVerkleinern** und **bildVergroessern** aufgerufen werden, wenn die Knöpfe geklickt werden.

Übung 13.49 Implementieren Sie die Methoden **bildVerkleinern** und **bildVergroessern**. Dazu müssen Sie ein neues **Farbbild** mit veränderter Größe erzeugen, die Bildpunkte aus dem aktuellen Bild in das neue kopieren (und dabei entsprechend skalieren) und dann das neue Bild als aktuelles Bild setzen. Am Ende Ihrer Methoden sollten Sie die Methode **pack** am **JFrame** aufrufen, damit die Komponenten an die geänderte Größe angepasst werden.

Übung 13.50 Alle Swing-Komponenten haben eine Methode **setEnabled** (**boolean**), mit der eine Komponente aktiviert und deaktiviert werden kann. Deaktivierte Komponenten erscheinen normalerweise in einem hellen Grau und reagieren nicht auf Eingaben. Ändern Sie Ihren Bildbetrachter so ab, dass die beiden Knöpfe zu Anfang deaktiviert sind. Wenn ein Bild geladen ist, sollen sie aktiviert sein, wenn es geschlossen ist, sollen sie wieder deaktiviert sein.

13.7.2 Rahmen

Zuletzt wollen wir unsere Benutzungsschnittstelle noch ein wenig mit internen Rahmen (*border*) aufpolieren. Rahmen können verwendet werden, um Komponenten zu gruppieren oder um lediglich für etwas Abstand zwischen ihnen zu sorgen. Jede Swing-Komponente kann einen Rahmen haben.

Einige Layout-Manager akzeptieren auch Konstruktorparameter zum Festlegen von Abständen, der Layout-Manager sorgt dann für den angeforderten Abstand zwischen seinen Komponenten.

Die am häufigsten verwendeten Rahmen sind **BevelBorder**, **CompoundBorder**, **EmptyBorder**, **EtchedBorder** und **TitledBorder**. Mit diesen sollten Sie sich vertraut machen.

Wir werden drei Dinge tun, um das Aussehen unserer GUI zu verbessern:

- etwas Abstand zu den Außenseiten eines Frame definieren,
- Abstand zwischen den Komponenten innerhalb eines Frame definieren und
- eine Linie um das Bild ziehen.

Der zugehörige Quelltext ist in Listing 13.16 zu sehen. Der Aufruf von **setBorder** an der Inhaltsfläche mit einem **EmptyBorder** als Parameter sorgt für etwas Abstand zu den Außenseiten des **JFrame**. Beachten Sie, dass wir nun an **contentPane** den

Cast-Operator mit `JPanel` anwenden, da der Supertyp `Container` keine Methode `setBorder` anbietet.⁵

```
JPanel contentPane = (JPanel)fenster.getContentPane();
contentPane.setBorder(new EmptyBorder(12, 12, 12, 12));

// Ein Layout mit hübschen Abständen definieren
contentPane.setLayout(new BorderLayout(6, 6));

// Die Bildfläche in der Mitte erzeugen
bildflaeche = new Bildflaeche();
bildflaeche.setBorder(new EtchedBorder());
contentPane.add(bildflaeche, BorderLayout.CENTER);
```

Listing 13.16

Abstände definieren.

Das Erzeugen eines `BorderLayout` mit zwei `int`-Parametern sorgt dafür, dass im Layout Abstände zwischen den Komponenten entstehen. Und schließlich führt das Setzen eines `EtchedBorder` an der `bildflaeche` zu einer Linie um das Bild, die dieses etwas eingelassen aussehen lässt. (Rahmen sind definiert im Paket `javax.swing.border` – wir müssen entsprechend eine `import`-Anweisung für dieses Paket einfügen.)

Alle besprochenen Verbesserungen sind in der nächsten Version dieser Anwendung im Zusatzmaterial enthalten: *Bildbetrachter3-0*. In dieser Version haben wir auch eine Funktion `SPEICHERN UNTER...` in das Dateimenü aufgenommen, damit Bilder auch wieder auf der Platte gespeichert werden können.

Wir haben außerdem einen weiteren Filter, den *Fischaugenfilter*, eingefügt, um Ihnen einen noch besseren Eindruck von den Möglichkeiten der Bildbearbeitung zu geben. Probieren Sie ihn aus. Er wirkt besonders gut bei Porträts.

13.8 Innere Klassen

Bis zu diesem Zeitpunkt haben wir Listener zur Ereignisverarbeitung mithilfe von Lambda-Ausdrücken implementiert. Das war möglich, weil die Listener-Interfaces funktionale Interfaces waren – d.h., sie besaßen nur eine einzige abstrakte Methode. Dies wird jedoch nicht bei allen Listener-Interfaces der Fall sein. Zum Beispiel bestehen die Interfaces `KeyListener`, `MouseListener` und `MouseMotionListener` aus dem Paket `java.awt.event` aus mehr als einer abstrakten Methode. Deshalb können wir für deren Implementierungen keine Lambda-Ausdrücke verwenden.

13.8.1 Benannte innere Klassen

Der offensichtliche Ansatz für solche Klassen besteht darin, eine separate Klasse zu definieren, die das benötigte Interface implementiert, und dann eine Instanz zu erzeugen, die als Listener-Objekt fungiert. Es gibt jedoch einige Nachteile bei diesem Ansatz, wenn wir einfach das Interface als eine normale Klasse innerhalb eines Projekts auf der gleichen Ebene wie alle anderen Klassen implementieren:

- Es gibt in der Regel eine sehr enge *Kopplung* zwischen einem Listener und dem Objekt, das die vielen Komponenten der gesamten GUI verwaltet. Der Listener wird häufig auf private Elemente auf den Zustand des GUI-Objekts

⁵ Die Anwendung des Cast-Operators funktioniert hier nur, weil der dynamische Typ der `ContentPane` bereits `JPanel` ist. Es ist nicht so, dass erst der Cast das `ContentPane`-Objekt in ein `JPanel` verwandeln würde!

zugreifen und diese verändern müssen. Dieser Kopplungsgrad ist nicht zu vermeiden, doch es wäre angenehmer, wenn wir einen Weg finden könnten, ihn unter den vielen Klassen eines Projekts klarer auszumachen.

- Wir haben gesehen, dass ein Lambda-Ausdruck vollständigen Zugang sogar zu den privaten Elementen der umschließenden GUI-Klasse hat – auf diese Art entsteht die enge Kopplung. Für eine externe Klasse wäre dies ohne zusätzliche sondierende und verändernde Methoden in der GUI-Klasse nicht möglich, die vermutlich ausschließlich zugunsten des Listeners zur Verfügung gestellt werden. Darüber hinaus könnte das Vorhandensein der Methoden eine unbeabsichtigte Kopplung zu anderen Klassen mit sich bringen.
- Wir erzeugen häufig nur eine einzige Instanz einer Listener-Klasse und eine vollständige externe Klasse erscheint für diese Art von Nutzung oft übertrieben.

Glücklicherweise stellt Java ein Konstrukt zur Verfügung, das all diese Probleme umgeht. Dieses Konstrukt haben wir bisher noch nicht behandelt: *innere Klassen*. Innere Klassen sind Klassen, die innerhalb einer anderen Klasse deklariert werden:

```
class UmschliessendeKlasse
{
    ...
    class InnereKlasse
    {
        ...
    }
}
```

Instanzen der inneren Klasse sind gebunden an Instanzen der umschließenden Klasse – sie können nur zusammen mit einer umschließenden Instanz existieren. Konzeptuell existieren sie *innerhalb* der umschließenden Instanz. Aus diesem Konstrukt sieht man direkt, dass die enge Kopplung zwischen innerer Listener-Klasse und der umschließenden GUI-Klasse eindeutig ist. Eine weitere Eigenschaft ist, dass Anweisungen in Methoden der inneren Klasse die privaten Datenfelder und Methoden der umschließenden Klasse sehen und auf diese zugreifen können – so wie es ein Lambda-Ausdruck kann. Die innere Klasse wird als ein Bestandteil der umschließenden Klasse angesehen, so wie die Methoden der umschließenden Klasse auch. Damit wird erfolgreich das zweite Problem angegangen, das wir oben beschrieben haben.

Wir können dieses Konstrukt nun benutzen, um ein **MouseListener** zu implementieren, beispielsweise innerhalb der Klasse **Bildbetrachter**. Die Struktur sieht folgendermaßen aus:

```
public class Bildbetrachter
{
    ...
    private class MouseHandler implements MouseListener
    {
        public voidMouseClicked(MouseEvent event)
        {
            // die Klick-Aktion ausführen
        }
        public void mouseEntered (MouseEvent event)
        {

```

```

        // die Eintreten-Aktion ausführen
    }
    ... weitere MouseListener-Methoden hier ausgelassen ...
}

```

(Per Konvention schreiben wir innere Klassen an das Ende der umschließenden Klasse – nach den Methoden.) Beachten Sie, dass wir die inneren Klassen mit privater Sichtbarkeit ausgestattet haben, um zu bekräftigen, dass sie eine Aufgabe durchführen, die sehr spezifisch für die **Bildbetrachter**-Klasse ist und nicht als unabhängig von dieser angesehen werden sollte.

Sobald wir eine innere Klasse definiert haben, können wir Instanzen von dieser auf die gleiche Art erzeugen, wie wir es mit Klassen bisher auch getan haben:

```

// Entdeckt Mausklicks auf dem Bild, wenn der Nutzer
// dieses bearbeiten möchte.
Bildflaeche.addMouseListener(new MouseHandler());

```

Beachten Sie einige typische Nutzungseigenschaften dieser inneren Listener-Klassen:

- Wir machen uns nicht die Mühe, die Instanzen in Variablen zu speichern – sie sind also eigentlich anonyme Objekte. Nur die Komponente, mit der sie verbunden sind, besitzt eine Referenz auf deren spezifisches Listener-Objekt, sodass sie die **Listener**-Methoden zur Ereignisverarbeitung aufrufen können.
- Wir erzeugen häufig nur ein einzelnes Objekt von jeder der inneren Klassen, da jede spezialisiert ist auf eine bestimmte Komponente innerhalb einer spezifischen GUI.

Diese Merkmale sind der Grund, dass wir im nächsten Abschnitt eine weitere Eigenschaft von Java untersuchen.

Innere Klassen können generell benutzt werden, um die Kohäsion in größeren Projekten zu verbessern. Das Projekt *Fuechse-und-Hasen* aus Kapitel 12 beispielsweise enthält eine Klasse **Simulationsansicht**, die eine innere Klasse **Feldansicht** hat. Sie können sich dieses Beispiel näher ansehen, um Ihr Verständnis zu vertiefen.

13.8.2 Anonyme innere Klassen

Die Lösung zur Implementierung von Interfaces mit mehreren Methoden auf Basis von inneren Klassen ist bereits ziemlich gut, aber wir wollen noch einen Schritt weitergehen: Wir möchten *anonyme innere Klassen* verwenden. Um dieses Konzept zu illustrieren, werden wir eine weitere Version des Bildbetrachters entwickeln: eine, die uns ein einfaches Bearbeiten der einzelnen Bildschirmpunkte erlaubt. Die Idee ist, dass der Benutzer irgendwo auf das Bild klicken kann, um die Farbe eines Bildschirmpunkts zu ändern. Das heißt, es müssen zwei Dinge gemacht werden:

- die Farbe auswählen, die im Bild platziert werden soll
- den Bildschirmpunkt auswählen, der ersetzt werden soll

Die Farbe wird über einen **JColorChooser**-Dialog ausgewählt, der erscheint, wenn der Benutzer eine Maustaste gleichzeitig mit der -Taste drückt. Der zu ersetzende Bildschirmpunkt wird ausgewählt, indem die Maus ohne -Taste geklickt

wird. Sobald eine Farbe ausgewählt wurde, werden die Bildschirmpunkte durch diese Farbe ersetzt, bis eine neue Farbe gewählt wird. Die Implementierung dazu finden Sie als *Bildbetrachter4-0*.

Natürlich ist ein Mausklick das einzige Ereignis, an dem wir interessiert sind, aber die Implementierung von **MouseListener** verlangt, dass wir fünf separate Methoden implementieren. Dies ist ärgerlich, da vier der fünf Methodenrumpfe leer sein werden. Wir werden oft ähnlichen Problemen begegnen, wenn wir andere Listener-Interfaces mit mehreren Methoden implementieren. Zum Glück haben die Entwickler der Java-API dieses Problem erkannt und sogenannte *no-op*-Implementierungen⁶ dieser Interfaces in Form von abstrakten **Adapter**-Klassen zur Verfügung gestellt, zum Beispiel **MouseAdapter** und **MouseMotionAdapter**. Dies bedeutet, dass wir häufig eine Subklasse einer **Adapter**-Klasse erzeugen können, anstatt das gesamte Interface zu implementieren, und nur die ein oder zwei Methoden überschreiben, die wir für eine bestimmte Aufgabe benötigen. In unserem Beispiel werden wir also eine innere Klasse anlegen, die eine Subklasse von **MouseAdapter** ist, und nur die **mousePressed**-Methode überschreiben.

Jetzt sind wir in der Lage, anonyme innere Klassen zu schreiben. Der relevante Code sieht folgendermaßen aus:

```
Bildflaeche = new Bildflaeche();
Bildflaeche.addMouseListener(new MouseAdapter() {
    public void mousePressed(MouseEvent e)
    {
        ... aktiv werden, wenn die Maus gedrückt wird ...
    }
});
```

Dieser Code sieht ausgesprochen mysteriös aus, wenn man ihn das erste Mal sieht, und es fällt Ihnen möglicherweise schwer, ihn zu interpretieren, selbst wenn Sie bisher alles in diesem Buch verstanden haben. Aber keine Angst – wir werden ihn ganz langsam auseinandernehmen.

Konzept

Anonyme innere Klassen

sind ein nützliches Konstrukt zur Implementierung von Event-Listnern, die keine funktionalen Interfaces sind.

Die Idee für dieses Konstrukt rührt von den Überlegungen her, dass wir jede innere Klasse nur genau einmal benutzen, um eine einzelne, unbenannte Instanz zu erzeugen. Für genau diese Situation bieten anonyme innere Klassen eine syntaktische Abkürzung: Sie erlauben uns, eine Klasse und auch gleich eine Instanz dieser Klasse in einem einzigen Schritt zu erzeugen. Der Effekt ist der gleiche wie bei der Version mit inneren Klassen, allerdings mit dem Unterschied, dass wir keine extra benannten Klassen definieren müssen und dass die Definition der Listener-Methoden noch dichter an der Registrierung des Listeners bei der GUI-Komponente steht.

Die farbliche Hervorhebung von zusammenhängenden Bereichen im BlueJ-Editor gibt uns einige Hinweise, die uns helfen können, diese Struktur besser zu verstehen (Abbildung 13.12) Die grüne Schattierung markiert eine Klasse, gelblich unterlegt sind die Methodendefinitionen und der weiße Hintergrund identifiziert einen Methodenrumpf. Wir stellen fest, dass der Rumpf der Methode **fensterErzeugen** eine sehr komprimierte (seltsam aussehende) Klassendefinition enthält, die eine Methodendefinition mit einem kurzen Rumpf aufweist.

⁶ Von englisch *no operation* (Nulloperation).

```

/**
 * Erzeuge das Swing-Fenster samt Inhalt.
 */
private void fensterErzeugen()
{
    fenster = new JFrame("Bildbetrachter");
    JPanel contentPane = (JPanel)fenster.getContentPane();
    contentPane.setBorder(new EmptyBorder(12, 12, 12, 12));

    menuezeileErzeugen(fenster);

    // Ein Layout mit hübschen Abständen definieren
    contentPane.setLayout(new BorderLayout(6, 6));

    // Die Bildfläche in der Mitte erzeugen
    bildflaeche = new Bildflaeche();
    bildflaeche.setBorder(new EtchedBorder());
    bildflaeche.addMouseListener(new MouseAdapter() {
        public void mousePressed(MouseEvent e) {handleMousePressed(e);}
    });

    contentPane.add(bildflaeche, BorderLayout.CENTER);

    // Zwei Labels oben und unten für den Dateinamen und Statusmeldungen

```

Abbildung 13.12

Farbliche Hervorhebung eines Bereichs mit einer anonymen inneren Klasse.

Wenn wir eine anonyme innere Klasse benutzen, dann erzeugen wir eine innere Klasse, *ohne sie zu benennen*, und erzeugen sofort ein einziges Exemplar dieser Klasse. Im obigen Quelltext geschieht dies durch das Code-Fragment

```

new MouseAdapter() {
    public void mousePressed(MouseEvent e) ... }
}

```

Eine anonyme innere Klasse wird erzeugt, indem ein Supertyp benannt wird (häufig eine abstrakte Klasse oder ein Interface, in diesem Fall **MouseListener**), gefolgt von einem Block mit Implementierungen der abstrakten Methoden des Supertyps oder Methoden, die wir überschreiben möchten. Dies sieht etwas ungewöhnlich aus, auch wenn es einige Gemeinsamkeiten zur Syntax der Lambda-Ausdrücke gibt.

In diesem Beispiel erzeugen wir eine neue Subklasse von **MouseListener**, die die Methode **mousePressed** überschreibt. Diese neue Subklasse bekommt keinen eigenen Namen. Stattdessen stellen wir das Schlüsselwort **new** voran und erzeugen so eine einzelne Instanz dieser Klasse. Diese einzelne Instanz ist ein **MouseListener**-Objekt (indirekt ein Subtyp von **MouseListener**) und kann so an die Methode **addMouseListener** einer GUI-Komponente übergeben werden. Jeder Subtyp eines so erzeugten **MouseListener**- oder **MouseListener**-Objekts repräsentiert eine eindeutige anonyme Klasse.

Genauso wie Lambda-Ausdrücke und benannte innere Klassen können anonyme innere Klassen auf die Datenfelder und Methoden der umschließenden Klasse zugreifen. Zusätzlich können sie, weil sie innerhalb einer Methode definiert wurden, auf die lokalen Variablen und die Parameter dieser Methode zugreifen.

Einige Beobachtungen zu anonymen inneren Klassen, die auch für Lambda-Ausdrücke gelten, müssen hervorgehoben werden. Diese Struktur ist angenehm kohärent und erweiterbar. Wenn wir einen zusätzlichen Menüeintrag benötigen, fügen wir einfach Code zum Erzeugen des Eintrags mitsamt Listener hinzu sowie eine Methode zur Behandlung der zugehörigen Funktion.

Es gilt aber, dass anonyme innere Klassen den Quelltext schwerer lesbar machen. Es wird dringend empfohlen, sie nur für sehr kurze Klassen und für etablierte Programmiermuster zu verwenden. Beispielsweise ist es fraglich, ob eine vollständige Implementierung des **MouseListener**-Interface als anonyme innere Klasse – angesichts der benötigten Codemenge – angemessen wäre. Eine benannte innere Klasse wäre wahrscheinlich in Bezug auf Kohäsion vorzuziehen. Für uns gilt, dass wir das Konstrukt in diesem Buch nur für die Ereignisverarbeitung benutzen.⁷

Übung 13.51 Sehen Sie sich die Implementierung des Bildpunkte-Editors in *Bildbetrachter4-0* noch einmal an. Überlegen Sie, ob die aktuellen Maus-Operationen, mit denen eine Farbe oder ein Bildpunkt ausgewählt wird, für einen Benutzer am geeignetsten sind. Ändern Sie sie, wenn Sie das Gefühl haben, dass sie es nicht sind.

Übung 13.52 Fügen Sie dem Projekt ein Menü hinzu, mit dem man im Bildpunkte-Editor eine „Pinzelgröße“ einstellen kann. Verwenden Sie die Pinzelgröße, um mehrere benachbarte Bildschirmpunkte mit jedem Mausklick zu bearbeiten.

Übung 13.53 Überschreiben Sie weitere Methoden der Klasse **MouseAdapter**, um Freihandzeichnen auf der Bildfläche zu erlauben. Überlegen Sie gründlich, ob Sie mit dem Listener als anonyme innere Klasse weiterarbeiten wollen oder ob Sie daraus eine benannte innere Klasse machen möchten.

13.9 Zusätzliche Erweiterungen

Das Programmieren von GUIs mit Swing ist ein riesiges Themengebiet. Swing bietet viele verschiedene Komponenten und viele verschiedene Container und Layout-Manager, jeweils mit zahlreichen Datenfeldern und Methoden.

Man braucht deshalb Zeit, um sich mit der gesamten Swing-Bibliothek vertraut zu werden, sicher mehr als ein paar Wochen. Normalerweise lesen wir uns weitere Details an, die wir noch nicht kannten, während wir an GUIs arbeiten, und werden auf diese Weise langsam zu Experten.

Das in diesem Kapitel beschriebene Beispiel ist, obwohl es eine Menge Details enthält, nur eine kurze Einführung in die GUI-Programmierung. Wir haben die meisten wichtigen Konzepte beschrieben, aber es gibt noch eine ganze Menge Funktionalität zu entdecken, die weit über den Rahmen dieses Buches hinausgeht. Es gibt verschiedene Quellen für weitergehende Informationen. Sie werden noch häufig die API-Dokumentation der Swing-Klassen nachschlagen müssen. Ohne diese geht es gar nicht. Es gibt außerdem viele Swing-/GUI-Tutorials, sowohl gedruckt als auch im Web.

⁷ Wenn Sie mehr über innere Klassen herausfinden wollen, dann sehen Sie sich die folgenden Abschnitte des Java-Tutorials an, das online verfügbar ist: <http://download.oracle.com/javase/tutorial/java/javaOO/nested.html> und <http://download.oracle.com/javase/tutorial/java/javaOO/innerclasses.html>.

Ein guter Startpunkt ist, wie bei vielem, das öffentlich verfügbare Java-Tutorial auf der Oracle-Website. Es enthält einen Abschnitt mit dem Titel *Creating a GUI with JFC/Swing* (<http://download.oracle.com/javase/tutorial/uiswing/index.html>).

In diesem Abschnitt gibt es einige spannende Unterabschnitte. Einer der nützlichsten ist vermutlich der Abschnitt *Using Swing Components* und darin der Unterabschnitt *How to ...*. Er enthält Einträge wie *How To Use Buttons, Check Boxes, and Radio Boxes*; *How to Use Labels*; *How to Make Dialogs*; *How to Use Panels* und so weiter.

In ähnlicher Weise enthält der Abschnitt *Laying Out Components Within a Container* einen Unterabschnitt *How to ...*, der alles über die verfügbaren Layout-Manager darstellt.

Übung 13.54 Finden Sie im Online-Java-Tutorial den Abschnitt *Creating a GUI with JFC/Swing* (die Abschnitte werden auf den Webseiten *trails* genannt). Erstellen Sie sich ein Lesezeichen in Ihrem Browser für diese Seite.

Übung 13.55 Was haben **CardLayout** und **GroupLayout** zu bieten, das sich von den bisher in diesem Kapitel diskutierten Layout-Managern unterscheidet?

Übung 13.56 Was ist ein *Slider*? Finden Sie eine Beschreibung und fassen Sie diese zusammen. Erstellen Sie ein kurzes Beispiel in Java, das einen Slider erzeugt und benutzt.

Übung 13.57 Was ist ein *Tabbed Pane*? Finden Sie eine Beschreibung und fassen Sie diese zusammen. Geben Sie Beispiele, bei denen ein Tabbed Pane nützlich sein kann.

Übung 13.58 Was ist ein *Spinner*? Finden Sie eine Beschreibung und fassen Sie diese zusammen.

Übung 13.59 Finden Sie die Demo-Anwendung *ProgressBarDemo*. Führen Sie sie auf Ihrem Computer aus. Beschreiben Sie, was Sie sehen.

Dies ist der Punkt, an dem wir die Diskussion des Bildbetrachter-Beispiels beenden. Es kann aber durch interessierte Leser in viele Richtungen erweitert werden. Mithilfe der Informationen aus dem Online-Tutorial können Sie zahlreiche Schnittstellen-Komponenten hinzufügen.

Die folgenden Übungen geben einige Anregungen; viele weitere sind denkbar.

Übung 13.60 Implementieren Sie eine Funktion *Rückgängig* in Ihrem Bildbetrachter. Diese Funktion nimmt die letzte Operation zurück.

Übung 13.61 Deaktivieren Sie alle Menüeinträge, die nicht benutzt werden können, wenn kein Bild geladen ist.

Übung 13.62 Implementieren Sie eine Funktion *Erneut laden*, die alle Änderungen am aktuellen Bild verwirft und es erneut von der Platte lädt.

Übung 13.63 Die Klasse `JMenu` ist tatsächlich eine Subklasse von `JMenuItem`. Dies bedeutet, dass geschachtelte Menüs erzeugt werden können, indem ein `JMenu` in ein anderes eingetragen wird. Fügen Sie ein Menü `AUSRICHTEN` in die Menüzeile ein. Fügen Sie in dieses ein Menü `ROTIEREN` ein, mit dem das Bild entweder um 90 oder 180 Grad gedreht werden kann, mit oder gegen den Uhrzeigersinn. Implementieren Sie diese Funktionalität. Das Menü `AUSRICHTEN` könnte auch Menüeinträge enthalten, die beispielsweise die bestehende Funktionalität zum Vergrößern und Verkleinern aufrufen.

Übung 13.64 Die Anwendung passt die Größe des Fensters immer wieder an, damit sichergestellt ist, dass das komplette Bild immer sichtbar ist. Ein sehr großes Fenster ist aber nicht immer wünschenswert. Lesen Sie die Dokumentation der Klasse `JScrollPane`. Statt die `Bildflaeche` direkt in die Inhaltsfläche einzufügen, fügen Sie sie in einen `JScrollPane` ein und dann den `JScrollPane` in die Inhaltsfläche. Laden Sie ein großes Bild und experimentieren Sie mit Größenänderungen am Fenster. Welcher Unterschied ergibt sich aus der Verwendung eines `JScrollPane`? Ermöglicht sie Ihnen die Anzeige von Bildern, die sonst zu groß für den Bildschirm wären?

Übung 13.65 Ändern Sie Ihre Anwendung so ab, dass sie mehrere Bilder gleichzeitig laden kann (aber immer nur ein Bild angezeigt wird). Fügen Sie dann ein Pop-up-Menü hinzu (Klasse `JComboBox`), mit dem das anzuzeigende Bild selektiert wird.

Übung 13.66 Verwenden Sie als Alternative zur `JComboBox` wie in **Übung 13.65** ein Registersteuerelement (Klasse `JTabbedPane`), um mehrere geöffnete Bilder zu verwalten.

Übung 13.67 Implementieren Sie eine Diashow-Funktion, bei der Sie ein Verzeichnis auswählen können und dann jedes Bild in diesem Verzeichnis für eine bestimmte Dauer angezeigt wird (beispielsweise fünf Sekunden).

Übung 13.68 Wenn Sie die Diashow implementiert haben, fügen Sie einen Schieberegler (Klasse `JSlider`) hinzu, mit dem Sie ein Bild der Show mit dem Regler auswählen können. Während die Diashow läuft, sollte der Schieberegler sich bewegen, um den Fortschritt anzuzeigen.

13.10 Ein weiteres Beispiel: der Musikplayer

In diesem Kapitel haben wir bisher ein Beispiel einer GUI-Anwendung ausführlich diskutiert. Wir wollen nun eine zweite Anwendung vorstellen, um aus einem weiteren Beispiel lernen zu können. Diese Anwendung führt uns einige weitere GUI-Komponenten vor.

Das zweite Beispiel ist ein Programm zum Abspielen von Musikdateien, also ein Musikplayer. Wir werden es hier nicht in voller Ausführlichkeit diskutieren. Es dient in erster Linie als Studienobjekt für Sie und als Quelle für Code-Fragmente, die Sie kopieren und verändern können. In diesem Kapitel werden wir nur auf einige Aspekte dieser Anwendung eingehen, die uns erwähnenswert erscheinen.

Übung 13.69 Öffnen Sie das Projekt *Musikplayer*. Erzeugen Sie eine Instanz der Klasse **MusikPlayerGUI** und experimentieren Sie mit der Anwendung.

Das Projekt *Musikplayer* bietet eine GUI für Klassen, die auf den *Musiksammlung*-Projekten aus Kapitel 4 basieren. Wie dort findet und spielt das Programm *mp3*-Dateien im Ordner *audio* unterhalb des Projektordners. Wenn Sie selbst über eigene Musikstücke im richtigen Format verfügen, sollten Sie sie abspielen können, wenn Sie sie im Ordner *audio* ablegen.

Der Musikplayer wird durch drei Klassen implementiert: **MusikPlayerGUI**, **MusicPlayer** und **MusicFilePlayer**. Wir wollen uns hier nur die erste ansehen. Die Klasse **MusicFilePlayer** kann im Prinzip als eine Bibliotheksklasse angesehen werden; Instanzen werden zusammen mit dem Namen der abzuspielenden *mp3*-Datei erzeugt. Machen Sie sich mit ihrer Schnittstelle vertraut, aber Sie brauchen ihre Implementierung nicht zu verstehen oder zu bearbeiten. (Natürlich können Sie sich bei Interesse diese Klasse genauer ansehen, aber sie benutzt einige Konzepte, die wir in diesem Buch nicht diskutieren werden.)

Es folgen einige erwähnenswerte Beobachtungen zu diesem Projekt.

Trennung von Modell und Ansicht

Diese Anwendung nimmt eine bessere Trennung zwischen Modell und Ansicht vor als das vorige Beispiel. Dies bedeutet, dass die Funktionalität der Anwendung (das Modell) sauber getrennt ist von ihrer Benutzungsschnittstelle (die GUI). Beide Teile, die Funktionalität und die Ansicht, können jeweils aus mehreren Klassen bestehen, aber jede Klasse sollte eindeutig einer der beiden Teile zugeordnet werden können, um eine klare Trennung zu erreichen. In unserem Beispiel besteht die Ansicht nur aus einer einzigen GUI-Klasse.

Die Trennung der Funktionalität einer Anwendung von ihrer Benutzungsschnittstelle bewirkt eine gute Kohäsion: Sie macht das Programm verständlicher, einfacher zu warten und einfacher an veränderte Anforderungen anpassbar (insbesondere an andere Benutzungsschnittstellen). So ist es beispielsweise vergleichsweise leicht, eine textbasierte Schnittstelle für den Musikplayer zu entwickeln, die letztlich **MusikPlayerGUI** ersetzt, aber **MusicPlayer** unverändert lässt.

Erben von JFrame

In diesem Beispiel demonstrieren wir eine andere häufig angewendete Vorgehensweise für das Konstruieren von Frames, die wir bereits am Anfang des Kapitels angesprochen haben. Unsere GUI-Klasse erzeugt kein Objekt der Klasse **JFrame**; sie erbt von der Klasse **JFrame**. Diese bewirkt, dass alle Methoden von **JFrame**, die wir aufrufen müssen (wie etwa **getContentPane**, **setJMenuBar**, **pack**, **setVisible** usw.), nun als interne (geerbte) Methoden aufrufbar sind.

Es gibt keinen besonderen Grund, die eine Variante (Benutzen einer Instanz von **JFrame**) gegenüber der anderen (Erben von **JFrame**) zu bevorzugen. Die Entscheidung kann nach persönlichen Vorlieben getroffen werden. In der Praxis sind beide Techniken weitverbreitet.

Unbewegte Bilder anzeigen

Sehr häufig wollen wir ein Bild in unserer GUI anzeigen. Die einfachste Art, dies zu tun, ist über einen **JLabel** in der Oberfläche, der eine Grafik anzeigt (ein **JLabel** kann entweder Text oder Grafik oder beides anzeigen). Der Musikplayer enthält ein Beispiel dafür. Die betreffende Quelltextzeile ist

```
JLabel bild = new JLabel(new ImageIcon("titel.jpg"));
```

Diese Anweisung lädt eine Bilddatei namens *titel.jpg* aus dem Projektverzeichnis, erzeugt ein Icon mit dem Bild und erzeugt dann einen **JLabel**, der das Icon anzeigt. (Der Begriff *Icon* scheint hier darauf hinzuweisen, dass wir nur mit kleinen Bildern umgehen, aber tatsächlich können die Bilder beliebige Größe haben.) Dies funktioniert für Bilder im JPEG-, GIF- und PNG-Format.

Combo-Boxen

Der Musikplayer enthält ein Beispiel für die Verwendung einer **JComboBox**. Eine Combo-Box ist eine Menge von Werten, von denen jeweils immer nur einer ausgewählt ist. Der gewählte Wert wird angezeigt und die Selektion kann über ein Popup-Menü vorgenommen werden. Im Musikplayer wird die Combo-Box verwendet, um eine bestimmte Reihenfolge auswählen zu können, z.B. nach Interpret, Titel usw.

Eine **JComboBox** kann auch editierbar sein. Die Werte sind dann nicht alle vordefiniert, sondern können durch den Benutzer eingegeben werden. In unserem Fall ist dies jedoch nicht so.

Listen

Das Programm enthält auch ein Beispiel für eine Liste (Klasse **JList**), für die Liste der Musikstücke. Eine Liste kann eine beliebige Anzahl von Werten halten und einer oder mehrere können selektiert sein. Die Werte in der Liste sind in diesem Fall Strings, aber andere Typen sind ebenso möglich. Eine Liste hat nicht automatisch einen Rollbalken.

Rollbalken

Ein weiterer Aspekt, der in diesem Beispiel demonstriert wird, ist die Verwendung von Rollbalken.

Rollbalken können über einen speziellen Container erzeugt werden – eine Instanz der Klasse **JScrollPane**. Beliebige GUI-Komponenten können in einen **JScrollPane** eingefügt werden, der für den Fall, dass die anzuzeigende Komponente zu groß für den zur Verfügung stehenden Platz ist, die notwendigen Rollbalken anzeigt.

In unserem Beispiel haben wir unsere Liste von Musikdateien in einen **JScrollPane** eingefügt. Der **JScrollPane** selbst wiederum wird in ihren Eltern-Container eingefügt. Die Rollbalken werden nur im Bedarfsfall angezeigt. Sie können dies ausprobieren, indem Sie entweder so viele Musikdateien hinzufügen, bis diese nicht mehr zusammen angezeigt werden können, oder indem Sie das Fenster so stark verkleinern, dass es kleiner ist als die aktuelle Liste.

Andere Elemente, die in diesem Beispiel demonstriert werden, sind ein Schieberegler (der nicht viel macht) und die Verwendung von Farben (aus der Liste) zur Veränderung des Aussehens einer Anwendung. Jedes GUI-Element bietet diverse Methoden an, mit deren Hilfe das Aussehen oder das Verhalten der Komponente geändert werden kann. Sie sollten sich die Dokumentation von Komponenten, die Sie interessieren, genauer ansehen und mit den Eigenschaften dieser Komponenten experimentieren.

Übung 13.70 Ändern Sie den Musikplayer so ab, dass er ein anderes Bild in der Mitte anzeigt. Suchen Sie sich ein Bild aus dem WWW oder erstellen Sie ein eigenes.

Übung 13.71 Ändern Sie die Farben der anderen Komponenten (Vorder- und Hintergrundfarbe) so ab, dass sie zu Ihrem Bild passen.

Übung 13.72 Fügen Sie eine neue Komponente hinzu, die nähere Angaben zu dem aktuell abgespielten Musikstück anzeigt.

Übung 13.73 Fügen Sie dem Musikplayer eine *reload*-Eigenschaft hinzu, die erneut den Ordner *audio* einliest. Dann können Sie eine neue Musikdatei in den Ordner einfügen und laden, ohne die Anwendung neu starten zu müssen.

Übung 13.74 Fügen Sie eine ÖFFNEN-Funktion in das DATEI-Menü ein. Wenn der Eintrag selektiert wird, öffnet sich ein Dateiauswahldialog, mit dem der Benutzer eine Musikdatei zum Laden auswählen kann. Wenn der Benutzer ein Verzeichnis auswählt, dann soll der Player alle Musikdateien aus diesem Verzeichnis laden (wie er es jetzt schon mit dem *audio*-Ordner tut).

Übung 13.75 Verändern Sie den Schieberegler so, dass Start und Ende (und möglicherweise weitere Zwischenstufen) mit Zahlen beschriftet sind. Der Start sollte Null sein, das Ende sollte die Länge des Musikstücks in Sekunden sein. Die Klasse **MusicPlayer** verfügt über eine Methode **gibLaenge**. Beachten Sie, dass der Schieberegler noch nicht funktioniert.

Übung 13.76 Verändern Sie den Musikplayer so, dass ein Doppelklick auf ein Element in der Liste der Musikstücke das Stück abspielt.

Übung 13.77 Verbessern Sie das Aussehen der Knöpfe. Alle Knöpfe, die zeitweilig keine Funktion haben, sollten während dieser Phasen ausgegraut sein. Sie sollten nur aktivierbar sein, wenn ihre Verwendung sinnvoll ist.

Übung 13.78 Die Anzeige der Musikstücke besteht zurzeit einfach aus einer **JList** von **String**-Objekten. Prüfen Sie, ob es irgendwelche Swing-Komponenten gibt, die besser geeignet sind. Gibt es beispielsweise eine Möglichkeit, eine Kopfzeile anzubieten und Interpret, Titel und andere Informationen der Musikstücke auszurichten? Implementieren Sie dies in Ihrer Version.

Übung 13.79 *Zusatzaufgabe.* Fügen Sie Quelltext ein, sodass sich der Schieberegler parallel zum Abspielen des Musikstücks bewegt.



Zusammenfassung

In diesem Kapitel haben wir eine Einführung in die GUI-Programmierung mit AWT und Swing gegeben. Wir haben die drei wichtigsten Konzepte erläutert: die Erzeugung von GUI-Komponenten, Layout und Ereignisverarbeitung.

Wir haben gesehen, dass der Aufbau einer GUI normalerweise mit der Erzeugung eines Fensters auf oberster Ebene beginnt, wie beispielsweise einem **JFrame**. Der Frame wird dann mit verschiedenen Komponenten befüllt, die dem Benutzer Informationen und Funktionalität anbieten. Unter anderem haben wir Menüs, Menüeinträge, Knöpfe, Labels und Rahmen kennengelernt.

Komponenten werden auf dem Bildschirm mithilfe von Containern und Layout-Managern arrangiert. Container halten Sammlungen von Komponenten und jeder Container hat einen Layout-Manager, der die Anordnung der Komponenten innerhalb des Bildschirmbereichs des Containers bestimmt. Das Verschachteln von Containern durch Kombinieren von verschiedenen Layout-Managern ist ein probates Mittel, um die gewünschte Kombination von Komponentengröße und Positionierung zu erreichen.

Interaktive Komponenten (solche, die auf Benutzereingaben reagieren können) erzeugen Ereignisse, wenn sie von Benutzern aktiviert werden. Andere Objekte können sich über das Auftreten solcher Ereignisse informieren lassen, indem sie Standard-Interfaces implementieren. Sie werden dadurch zu Listener-Objekten. Wenn ein Listener-Objekt benachrichtigt wird, kann es die angemessenen Maßnahmen ergreifen, um mit dem Ereignis umzugehen. Event-Listener werden häufig mithilfe von Lambda-Ausdrücken implementiert.

Wir haben anonyme innere Klassen als eine alternative modulare, erweiterbare Technik für das Schreiben von Ereignis-Listnern vorgestellt. Diese Klassen sind besonders nützlich, wenn ein Listener-Interface kein funktionales Interface ist (es hat mehr als eine abstrakte Methode) und nicht mithilfe von Lambda-Ausdrücken implementiert werden kann.

Und schließlich haben wir einen Hinweis auf das Online-Tutorial gegeben, mit dessen Hilfe Details nachgeschlagen werden können, die in diesem Kapitel nicht behandelt wurden.

NEUE BEGRIFFE IN DIESEM KAPITEL

GUI, AWT, Swing, Komponente, Layout, Ereignis (*event*), Ereignisverarbeitung, Event-Listener, Frame, Menüzeile, Menü, Menüeintrag, Inhaltsfläche (*content pane*), modaler Dialog, anonyme innere Klasse

Zusammenfassung der Konzepte

- **Komponenten** Eine GUI wird erzeugt, indem Komponenten auf dem Bildschirm arrangiert werden. Komponenten werden durch Objekte repräsentiert.
- **Layout** Das Layout der Komponenten, also ihre Anordnung auf dem Bildschirm, wird durch Layout-Manager kontrolliert.
- **Ereignisverarbeitung** Der Begriff *Ereignisverarbeitung* bezeichnet die Aufgabe, auf Benutzereingaben wie Mausklicks oder Tastendrucke zu reagieren.
- **Bildformate** Digitale Bilder können in unterschiedlichen Formaten gespeichert werden. Die Unterschiede wirken sich primär auf die Größe und den Informationsgehalt aus.
- **Menüzeile, Inhaltsfläche** Komponenten werden in einem Fenster platziert, indem sie entweder in die Menüzeile (*menu bar*) oder die Inhaltsfläche (*content pane*) eingefügt werden.
- **Event-Listener** Ein Objekt kann auf Ereignisse „lauschen“, indem es ein Event-Listener-Interface implementiert.
- **Anonyme innere Klassen** Anonyme innere Klassen sind ein nützliches Konstrukt zur Implementierung von Event-Listnern, die keine funktionalen Interfaces sind.



Übung 13.80 Versehen Sie das Projekt *Zuul* aus Kapitel 8 mit einer GUI. Für jeden Raum sollte es ein passendes Bild geben, das angezeigt wird, wenn ein Spieler den Raum betritt. Es sollte einen nicht editierbaren Textbereich geben, in dem Text ausgegeben werden kann. Für die Eingabe von Kommandos gibt es verschiedene Möglichkeiten: Sie können sie textbasiert lassen und ein Textfeld (Klasse `JTextField`) zur Eingabe vorsehen, oder Sie können Kommandos durch Knöpfe auslösen lassen.

Übung 13.81 Fügen Sie dem *Zuul*-Projekt Töne hinzu. Sie können individuelle Geräusche mit Räumen, Gegenständen oder Figuren verbinden.

Übung 13.82 Entwerfen und bauen Sie eine GUI für einen Texteditor. Benutzern sollte ermöglicht werden, Text einzugeben, zu ändern, sich im Text zu bewegen etc. Überlegen Sie sich Funktionen zum Formatieren (Zeichensatz, Zeichengröße und -varianten wie kursiv und fett) und eine Funktion zum Zählen der Zeichen und Wörter. Die Funktion zum Laden und Speichern von Texten brauchen Sie noch nicht zu implementieren – damit können Sie warten, bis Sie das nächste Kapitel gelesen haben.



KAPITEL

14

Fehlerbehandlung

Lernziele

Zentrale Konzepte in diesem Kapitel: defensive Programmierung, Fehler melden, Exceptions auslösen und behandeln, einfache Dateiverarbeitung

Java-Konstrukte in diesem Kapitel: `TreeMap`, `TreeSet`, `SortedMap`, `assert`, `Exception`, `throw`, `throws`, `try`, `catch`, `File`, `FileReader`, `FileWriter`, `Path`, `Scanner`, `Stream`

In Kapitel 9 haben wir erwähnt, dass logische Programmfehler schwieriger zu finden sind als syntaktische Fehler, weil ein Compiler bei logischen Fehlern nicht helfen kann. Logische Fehler entstehen aus unterschiedlichen Gründen, die sich oft überschneiden:

- Die Lösung für ein Problem wurde nicht korrekt implementiert. Beispielsweise könnte bei einer Statistikaufgabe für eine Menge an Werten statt des Medians (des „mittleren“ Werts) fälschlicherweise der Durchschnittswert berechnet werden.
- Ein Objekt könnte um eine Dienstleistung gebeten werden, die es nicht erbringen kann. Beispielsweise könnte die `get`-Methode eines Sammlungsobjekts mit einem Index aufgerufen werden, der außerhalb des gültigen Bereichs liegt.
- Ein Objekt könnte auf eine Weise benutzt werden, die der Entwerfer der Klasse so nicht vorhergesehen hat und die das Objekt in einen inkonsistenten oder unangemessenen Zustand versetzt. Das kann leicht passieren, wenn eine Klasse in einem anderen als ihrem ursprünglich geplanten Zusammenhang wiederverwendet wird, beispielsweise durch Vererbung.

Obwohl wir mit den in Kapitel 9 diskutierten Teststrategien viele logische Fehler finden und beheben können, bevor ein Programm zum Einsatz kommt, zeigt die Erfahrung doch auch, dass es weiterhin zu Programmfehlern kommen wird. Selbst das bestgetestete Programm kann scheitern, wenn Umstände zum Tragen kommen, die außerhalb des Einflussbereichs des Entwicklers liegen. Nehmen Sie den Fall eines Webbrowsers, der eine nicht existierende Seite anzeigen soll, oder eines Programms, das Daten in ein Dateisystem schreiben soll, dessen Speicher ausgeschöpft ist. Solche Probleme sind nicht die Folge logischer Programmierfehler, können aber ein Programm leicht scheitern lassen, wenn dieser Problemfall nicht vorhergesehen wurde.

In diesem Kapitel werden wir betrachten, wie Fehlersituationen vorausgesehen werden können und wie im laufenden Programm auf solche Fehlersituationen reagiert werden kann. Außerdem werden wir aufzeigen, wie auftretende Fehler gemeldet